



Property Hub

A Real Estate Ai Agent

Presented By

1. Ahmed Farag Ahmed
2. Mahmoud Mohamed AbdElsalam
3. AbdElrahman Amr Abdeslam
4. Ahmed Mohamed Ibrahim
5. Hassan Ahmed Mohamed

Supervised By:

Dr. Yasser Fakhry

2026

Table of Contents

1. Abstract	5
2. Introduction	7
2.1 Motivation	7
2.2 Problem Statement	8
2.3 Goals of the Project	9
2.4 Existing Solutions	10
2.5 Features Matrix	11
2.6 Development Methodology	12
3. Requirements Analysis	13
3.1 Functional Requirements	13
3.1.1 User Authentication and Authorization	13
3.1.2 Property Management	13
3.1.3 Property Recommendation System	14
3.1.4 Property Price Prediction	14
3.1.5 User Interaction Tracking	14
3.1.6 Analytics Dashboard	14
3.1.7 RESTful API Services	14
3.1.8 Frontend User Interface	15
3.2 Non-Functional Requirements	15
3.2.1 Performance Requirements	15
3.2.2 Security Requirements	15
3.2.3 Scalability Requirements	15
3.2.4 Availability Requirements	15
3.2.5 Maintainability Requirements	16
3.2.6 Usability Requirements	16
3.2.7 Reliability Requirements	16
3.3 Use Case Diagram	16
4. System Design	19
4.1 System Architecture	19
4.2 Entity Relationship Diagram (ERD)	21
4.3 UML Class Diagram	24
4.4 Sequence Diagrams	30
4.4.1 User Authentication Flow	31
4.4.2 Recommendation Generation Flow	32
4.4.3 Property Price Prediction Flow	33

4.5 Data Flow Diagrams	34
4.5.1 Level 0 Data Flow Diagram (Context Diagram)	35
4.5.2 Level 1 Data Flow Diagram	36
4.6 Machine Learning Workflow	38
4.7 Machine Learning Workflow	40
4.7.1 Data Collection	41
4.7.2 Data Preprocessing	41
4.7.3 Feature Engineering	42
4.7.4 Model Training	43
4.7.5 Model Evaluation	44
4.7.6 Deployment and Prediction	45
4.8 API Endpoints Design	46
4.8.1 Authentication API	47
4.8.2 Property Management APIs	50
4.8.3 Recommendation APIs	52
4.8.4 Price Prediction APIs	54
4.8.5 Analytics APIs	55
4.9 Dataset Description	56
4.9.1 Dataset Features	57
4.9.2 Data Preprocessing Operations	58
4.9.3 Training and Testing Split	59
4.9.4 Machine Learning Algorithms Used	59
4.9.5 Model Evaluation Results	63
5. Implementation	64
5.1 Development Environment and Technologies	65
5.2 Frontend Implementation	69
5.3 Backend Implementation	70
5.4 Recommendation System Implementation	72
5.5 Price Prediction System Implementation	75
5.6 Database Implementation	77
5.7 Machine Learning Integration	78
5.8 Authentication and Security	79
5.9 System Testing	80
5.10 Deployment and System Execution	82
5.11 Challenges and Solutions	83
5.12 Summary	85
6. Results and Discussion	86
6.1 Recommendation System Results	86

6.2 Property Price Prediction Results	88
6.2.1 Prediction Evaluation Metrics	90
6.3 Backend Performance Results	90
6.4 User Interface Results	91
6.5 Discussion	91
6.6 Summary	92
7. Conclusion and Future Work	93
7.1 Conclusion	93
7.2 Achieved Objectives	94
7.3 Future Work	95
7.4 Final Summary	96
8. References	97
9. Appendices	99

List Of Figure:

Figure 3.3: Use Case	18
Figure 4.2: ERD	24
Figure 4.3: UML	30
Figure 4.4.1: Sequence Diagrams	31
Figure 4.4.2: Recommendation Generation Flow	33
Figure 4.4.3: Property Price Prediction Flow	34
Figure 4.5.1 Level 0 Data Flow	36
Figure 4.5.2 Level 1 Data Flow	37
Figure 4.6 System Architecture	40
Figure 4.7 Machine Learning Workflow	46

1. Abstract

The real estate sector has experienced significant digital transformation in recent years, resulting in an increasing demand for intelligent platforms capable of delivering Personalized experiences and data-driven decision support. Traditional property listing systems often provide static search functionalities and generic recommendations, making it difficult for users to efficiently discover relevant properties and accurately evaluate market opportunities.

This project introduces **Property Hub**, an AI-powered real estate platform that integrates advanced machine learning techniques with modern web technologies to enhance property discovery, recommendation, and valuation processes. The platform was designed to provide a comprehensive environment where users can browse properties, receive personalized recommendations, predict property prices, and analyze real estate information through an intelligent and scalable architecture.

The proposed system adopts a multi-layer architecture consisting of a responsive frontend interface, a FastAPI-based backend, a MySQL database, and dedicated machine learning services. A hybrid recommendation framework was developed to generate personalized property suggestions by combining user behavior analysis, content-based filtering, interaction tracking, candidate generation, ranking algorithms, and preference modeling techniques. User activities such as property views, clicks, searches, and saved properties are continuously analyzed to construct dynamic user profiles and improve recommendation relevance.

In addition to recommendation services, the platform incorporates machine learning-based property price prediction models trained on real estate datasets and engineered property features. Multiple regression algorithms were evaluated to estimate property values accurately and support users during investment and purchasing decisions. The system also includes analytics modules for monitoring user engagement, recommendation performance, and overall platform activity.

To ensure scalability, maintainability, and security, the platform was implemented using a modular service-oriented architecture with RESTful APIs, secure authentication and authorization mechanisms based on JSON Web Tokens (JWT), and efficient database management through SQLAlchemy and MySQL. The machine learning components were fully integrated into the backend environment, enabling real-time recommendation generation and property valuation services.

Experimental evaluation demonstrated the effectiveness of integrating behavioral analytics, intelligent recommendation techniques, and predictive machine learning models within a unified real estate platform. The proposed solution improves recommendation quality, enhances user experience, supports informed decision-making, and establishes a strong foundation for future AI-driven real estate applications.

Keywords: Artificial Intelligence, Real Estate Technology, Hybrid Recommendation System, Machine Learning, Property Price Prediction, Behavioral Analytics, FastAPI, MySQL, Learning-to-Rank, Intelligent Decision Support Systems.

2.Introduction

2.1 Motivation :

The increasing dependence on digital platforms in the real estate sector has created a growing demand for intelligent systems capable of simplifying the property search and decision-making process. Traditional real estate websites mainly focus on displaying property listings without providing users with personalized assistance or analytical insights. As a result, users often spend significant time searching through large numbers of properties without receiving recommendations that match their preferences and needs.

In addition, estimating property prices accurately remains one of the major challenges in the real estate market. Property values are affected by multiple dynamic factors such as location, size, amenities, and market trends. Conventional systems do not provide intelligent tools that help users evaluate whether a property price is reasonable or not.

Recent advances in Machine Learning and Artificial Intelligence have enabled the development of intelligent recommendation systems and predictive analytics solutions capable of analyzing user behavior and large datasets efficiently. Integrating these technologies into real estate platforms can significantly improve the user experience and support better decision-making.

The motivation behind developing Property Hub is to create an intelligent and user-centered real estate platform that combines modern web technologies with machine learning techniques to provide personalized recommendations, accurate property price prediction, and advanced analytical capabilities.

"Furthermore, the platform leverages user interaction tracking, behavioral analytics, and hybrid recommendation techniques to continuously adapt recommendations according to user preferences and evolving interests."

2.2 Problem Statement :

The real estate market contains a massive number of property listings with different prices, locations, and features, making it difficult for users to identify suitable properties efficiently. Most existing real estate platforms provide only basic search and filtering functionalities without considering user preferences, browsing behavior, or personalized recommendations.

Another major issue is the lack of intelligent price estimation tools. Property prices vary significantly depending on several factors such as area, location, number of rooms, facilities, and market conditions. Users often face difficulties determining whether a property price is fair and appropriate compared to similar properties.

Furthermore, many existing systems do not provide sufficient analytical tools for administrators to monitor user activities, property engagement, and recommendation performance. The absence of interaction tracking and behavioral analysis limits the ability of systems to improve recommendation quality and user satisfaction over time.

Additionally, conventional recommendation approaches often struggle to adapt dynamically to evolving user interests and interaction patterns. Without behavioral analytics and intelligent ranking mechanisms, recommendation quality may decrease over time, leading to reduced user engagement and satisfaction. Therefore, modern real estate platforms require advanced recommendation architectures capable of learning from user interactions and continuously refining recommendation relevance.

Therefore, there is a need for a smart real estate platform capable of integrating machine learning techniques with modern web technologies to provide personalized recommendations, accurate property price prediction, user behavior analysis, and advanced analytics in a scalable and secure environment.

2.3 Goals of the Project

The main goal of Property Hub is to develop a smart real estate platform that improves the process of searching, analyzing, and recommending properties using machine learning technologies and modern web development frameworks.

The project aims to provide users with personalized property recommendations based on their interactions, browsing behavior, and preferences. In addition, the system seeks to estimate property prices accurately using machine learning prediction models trained on real estate datasets and property-related features.

Another important objective of the project is to provide administrators with analytical dashboards and monitoring tools that help analyze user engagement, recommendation performance, and system activity. The project also focuses on implementing secure authentication mechanisms, scalable backend services, and efficient frontend interfaces to ensure a smooth and reliable user experience.

The specific goals of the project include:

- Developing a complete web-based real estate platform.
- Implementing a machine learning recommendation system.
- Building a property price prediction model.
- Tracking user interactions and behavioral activities.
- Providing analytical dashboards for administrators.
- Designing RESTful APIs using FastAPI.
- Implementing secure authentication and authorization mechanisms.
- Improving property search efficiency and recommendation relevance.
- Providing scalable and maintainable system architecture.

2.4 Currently Available Solutions (Existing Applications)

Several online real estate platforms currently exist and provide property listing and search functionalities. These platforms allow users to browse available properties, compare prices, and communicate with sellers or real estate agencies. However, most existing solutions focus primarily on displaying listings and basic filtering options without offering advanced intelligent services such as personalized recommendations or accurate price prediction.

One of the most popular real estate platforms in Egypt is Aqarmap, which provides property listings for sale and rent across different cities and regions. The platform offers filtering options based on price, location, and property type. However, the system mainly depends on manual searching and does not provide intelligent recommendation mechanisms based on user behavior or interaction history [1].

Another widely used platform is Dubizzle (formerly OLX Egypt), which allows users to post and browse real estate advertisements. Although the platform supports a large number of listings and user interactions, it lacks advanced analytical tools and machine learning-based recommendation systems that improve user experience and decision-making [2].

International platforms such as Zillow provide additional analytical features including property price estimation and market insights. However, these systems are mainly designed for international real estate markets and may not fully address local market characteristics or user preferences within the Egyptian real estate environment [3].

Most existing systems share several limitations:

- Limited personalization and recommendation capabilities.
- Weak integration of machine learning technologies.
- Lack of user behavior analysis and interaction tracking.
- Limited analytical dashboards for administrators.
- Dependence on manual filtering and searching processes.

Property Hub aims to overcome these limitations by integrating machine learning recommendation systems, property price prediction models, behavioral analytics, and scalable backend services into a unified intelligent real estate platform.

2.5 Features Matrix

The following table compares Property Hub with existing real estate platforms based on the main system features and intelligent capabilities.

Features	Property Hub	Aqarmap	Dubizzle	Zillow
Property Listings	✓	✓	✓	✓
Property Search and Filtering	✓	✓	✓	✓
Personalized Recommendations	✓	X	X	Limited
Machine Learning Integration	✓	X	X	Partial
Property Price Prediction	✓	X	X	✓
User Behavior Tracking	✓	X	X	Limited
Analytics Dashboard	✓	X	X	Limited
RESTful APIs	✓	Limited	Limited	✓
Scalable Backend Architecture	✓	Limited	Limited	✓
Recommendation System	✓	X	X	Partial
Interaction Analytics	✓	X	X	Limited

The comparison demonstrates that Property Hub provides additional intelligent functionalities that are not fully available in most traditional real estate platforms. The integration of machine learning techniques, analytics dashboards, and recommendation systems enhances the overall user experience and supports more efficient property exploration and decision-making[1][2][3].

2.6 Development Methodology

The development of Property Hub followed an Agile Software Development methodology to ensure flexibility, continuous improvement, and efficient collaboration among team members throughout the project lifecycle.

The Agile [4] methodology was selected because the project involved multiple interconnected components, including frontend development, backend services, database integration, machine learning pipelines, recommendation systems, and analytics dashboards. Agile development allowed the team to divide the project into smaller manageable phases and continuously improve system functionalities through iterative development and testing.

The development process was organized into iterative sprints. Each sprint focused on a specific system component such as frontend development, backend services, database integration, recommendation modules, or machine learning models. Continuous testing and feedback were incorporated throughout the development lifecycle to ensure incremental improvement and system stability.

The project development process was divided into several stages:

- Requirements Analysis

During this stage, the team identified system requirements, analyzed existing real estate platforms, and determined the main functionalities required for the proposed system.

- System Design

The system architecture, database schema, UML diagrams, ERD models, APIs, and machine learning workflows were designed during this phase.

- Implementation

Frontend interfaces, backend APIs, authentication mechanisms, machine learning models, and analytics modules were implemented incrementally using iterative development cycles.

- Testing and Evaluation

Each module was tested individually and then integrated into the complete system. API testing, frontend testing, backend testing, and machine learning evaluation were performed to ensure system reliability and performance.

- Deployment and Documentation

After completing implementation and testing, the system was prepared for deployment and the final technical documentation was developed.

The Agile approach helped the development team adapt to changing requirements, improve system quality continuously, and integrate machine learning services efficiently within the overall platform architecture.

3. Requirements Analysis

The requirements analysis phase focused on identifying the system functionalities, user expectations, and technical requirements necessary for developing an intelligent real estate platform. This phase played a critical role in defining the scope of the project and ensuring that the proposed system satisfies both user needs and business objectives.

The analysis process included studying existing real estate platforms, identifying their limitations, collecting user requirements, and determining the necessary intelligent features required to improve the property search experience. Special attention was given to machine learning integration, recommendation systems, property price prediction, and user interaction analytics.

The requirements were categorized into functional and non-functional requirements to provide a clear understanding of system behavior, performance expectations, and architectural constraints.

3.1 Functional Requirements

Functional requirements describe the core services and operations that the system must perform to satisfy user and administrator needs.

3.1.1 User Authentication and Authorization

The system shall allow users to:

- Register new accounts.
- Log into the system securely.
- Access protected resources using authenticated sessions.
- Update profile information.
- Manage personal account settings.

The authentication mechanism is implemented using JWT (JSON Web Tokens) to ensure secure communication and protected API access.

3.1.2 Property Management

The system shall allow users and administrators to:

- View property listings.
- Add new properties.
- Update existing property information.
- Delete property records.
- Filter and search properties based on multiple criteria.

3.1.3 Property Recommendation System

The system shall:

- Analyze user interactions and preferences.
- Generate personalized property recommendations [5].
- Recommend similar properties based on user behavior and property features [6].
- Continuously improve recommendations using interaction tracking data.

3.1.4 Property Price Prediction

The system shall:

- Predict property prices using machine learning models [7].
- Accept property-related features as input parameters.
- Return estimated property values based on trained predictive models.

3.1.5 User Interaction Tracking

The system shall:

- Track property views and clicks.
- Monitor user browsing activities.
- Store interaction history for recommendation improvement.
- Analyze user engagement behavior.

3.1.6 Analytics Dashboard

The system shall provide administrators with:

- User activity statistics.
- Property engagement analytics.
- Recommendation performance indicators.
- System usage monitoring tools.

3.1.7 RESTful API Services

The backend system shall expose REST APIs for:

- Authentication services.
- Property management operations.
- Recommendation requests.
- Price prediction services.
- Analytics and interaction tracking.

3.1.8 Frontend User Interface

The frontend shall provide:

- Responsive web pages.
- Property browsing interfaces.
- Recommendation sections.
- Prediction forms and result pages.
- Analytics dashboards.
- User profile pages.

3.2 Non-Functional Requirements

Non-functional requirements describe the quality attributes and operational constraints of the system.

3.2.1 Performance Requirements

The system should provide fast API response times and efficient data processing capabilities. Machine learning predictions and recommendation generation should be performed within acceptable response times to maintain a smooth user experience.

3.2.2 Security Requirements

The system must ensure secure authentication and authorization using JWT tokens and password hashing techniques. Sensitive user information must be protected against unauthorized access and common web vulnerabilities.

3.2.3 Scalability Requirements

The system architecture should support future scalability to accommodate increasing numbers of users, properties, and interaction records. The modular backend architecture allows future integration of additional machine learning models and cloud deployment services.

3.2.4 Availability Requirements

The platform should maintain high system availability and reliable access to frontend interfaces, backend APIs, and machine learning services.

3.2.5 Maintainability Requirements

The system should be designed using modular components to simplify debugging, maintenance, and future development. Separate frontend, backend, and machine learning layers improve maintainability and code organization.

3.2.6 Usability Requirements

The frontend interfaces should provide a simple and user-friendly experience that allows users to browse properties, receive recommendations, and predict property prices efficiently.

3.2.7 Reliability Requirements

The system should ensure consistent recommendation generation, accurate prediction outputs, and stable API operations under different usage conditions.

3.3 Use Case Diagram

The Use Case Diagram represents the interaction between the main actors and the functionalities provided by the Property Hub system. The system contains three primary actors: the User, the Administrator, and the Machine Learning Engine.

The User interacts with the platform to browse properties, search listings, receive personalized recommendations, predict property prices, and manage personal account information. The Administrator is responsible for managing properties, monitoring user activities, viewing analytics reports, and supervising overall system operations. The Machine Learning Engine acts as an intelligent component responsible for generating recommendations, predicting property prices [8][9], and analyzing user behavior based on interaction history.

The Use Case Diagram demonstrates how machine learning services are integrated with the web application [10][11][12] to provide intelligent functionalities that improve user experience and decision-making.

User Use Cases

The user can perform the following operations:

- Register a new account.
- Login securely into the system.
- Browse available property listings.
- Search and filter properties based on multiple criteria.
- View detailed property information.
- Receive personalized property recommendations.
- Predict property prices using machine learning models.
- Save and manage favorite properties.
- View analytics dashboard information.
- Update personal profile information.

Administrator Use Cases

The administrator can perform the following operations:

- Manage property listings (Create, Update, Delete).
- Manage system users.
- Monitor user activities and interactions.
- View analytics and system reports.
- Monitor recommendation system performance.
- Configure system settings and platform management tools.

Machine Learning Engine Use Cases

The machine learning engine is responsible for:

- Generating personalized recommendations [11].
- Predicting property prices [7][8].
- Analyzing user behavior and interaction patterns.
- Processing interaction data to improve recommendation quality.

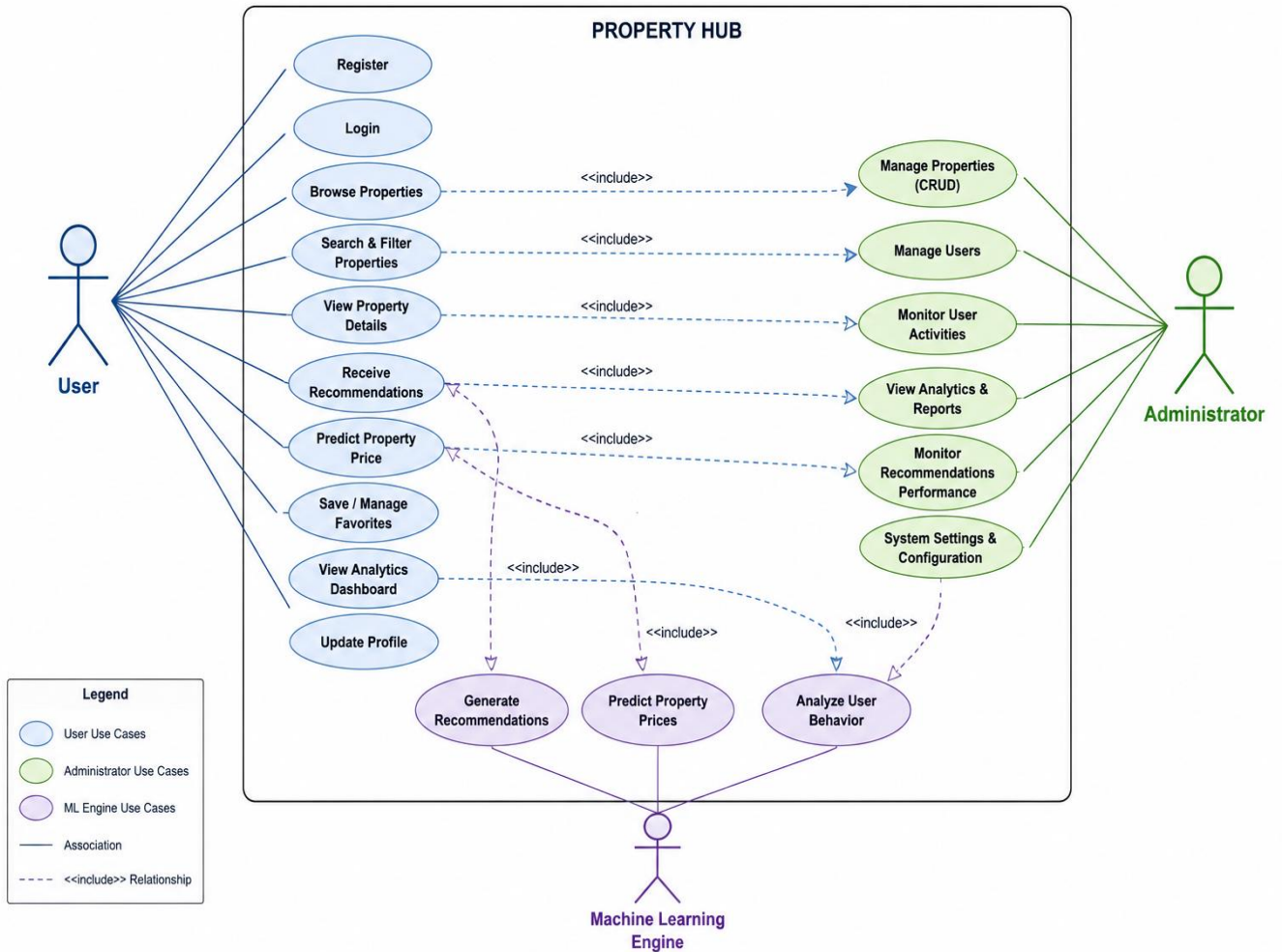


Figure 3.3 illustrates the complete Use Case Diagram for the Property Hub system and the relationships between users, administrators, and intelligent machine learning services.

4. System Design

This chapter presents the design aspects of Property Hub, including system architecture, sequence diagrams, entity-relationship diagrams, data flow diagrams, machine learning workflows, and UML class diagrams. The design phase focused on creating a scalable and modular architecture capable of integrating frontend interfaces, backend services, databases, and machine learning models efficiently.

The proposed system follows a layered architecture in which each layer performs a specific responsibility while maintaining clear communication with other components. This modular structure simplifies system maintenance, improves scalability, and facilitates future enhancements.

The main design objectives of the system include:

- Providing scalable backend services.
- Supporting machine learning integration.
- Improving maintainability through modular architecture.
- Ensuring secure communication between components.
- Simplifying frontend-backend interaction using REST APIs.
- Supporting analytical and recommendation functionalities.

4.1 System overview

The architecture of Property Hub follows a client-server model integrated with machine learning services [10] and analytics modules. The system consists of four major layers: the Frontend Layer, Backend Layer, Database Layer, and Machine Learning Layer.

Frontend Layer

The frontend layer was implemented using HTML, CSS, and JavaScript to provide responsive and interactive user interfaces. The frontend communicates with backend APIs through asynchronous HTTP requests and displays property listings, recommendations, analytics, and prediction results [13].

The frontend interfaces include:

- Login and Registration Pages
- Property Listings Page
- Recommendation Interface
- Price Prediction Interface
- Analytics Dashboard
- User Profile Management

Backend Layer

The backend layer was implemented using FastAPI due to its high performance and asynchronous processing capabilities. The backend handles authentication [12], business logic, API routing, interaction tracking, recommendation requests, and machine learning integration.

The backend architecture includes:

- Authentication Router
- Properties Router
- Analytics Router
- Interactions Router
- Machine Learning Router
- User Profile Router

Database Layer

The database layer stores user information, property records, interaction history [14], analytics data, and recommendation-related information. The database supports efficient querying and interaction tracking for recommendation generation and analytical processing .

Stored data includes:

- User Accounts
- Property Listings
- User Interactions
- Recommendation Logs
- User Behavior Data

Machine Learning Layer

The machine learning layer is responsible for recommendation generation and property price prediction[8][11]. This layer processes user behavior data, extracts features, trains machine learning models, and provides prediction and recommendation services through backend APIs.

The machine learning module includes:

- Recommendation Engine
- Candidate Generation Module
- User Behavior Analytics
- Learning-to-Rank Module
- Feature Engineering Pipeline
- Property Price Prediction Model
- Evaluation and Scoring Components

The modular architecture allows independent development and maintenance of each system layer while ensuring efficient communication between frontend interfaces, backend APIs, databases, and machine learning services.

4.2 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram (ERD) illustrates the database structure of the Property Hub system and the relationships between the core system entities. The database was designed to support property management, user authentication, recommendation generation, interaction tracking, analytics processing, and machine learning functionalities [10][14].

The proposed database architecture follows a relational structure that enables efficient data storage, retrieval, and analysis while supporting scalable backend services and intelligent recommendation workflows [10].

The main entities of the system include Users, Properties, Interactions, Recommendations, Price Predictions, Analytics, Search History, Categories, and Property Images.

Main Entities

User Entity

The User entity stores account and authentication-related information for all system users.

Main attributes include:

- User ID
- Username
- Email Address
- Password Hash
- Phone Number

- is admin
- is verification

Property Entity

The Property entity stores all information related to real estate listings available on the platform.

Main attributes include:

- Property ID
 - Property Title
 - Description
 - Property Type
 - Price
 - Area sqm
 - Location
 - City
 - Bedrooms
 - Bathrooms
 - Amenities
-
- update at

Interaction Entity

The Interaction entity records all user activities performed on property listings. These interactions are used by the recommendation engine and analytics services.

Main attributes include:

- Interaction ID
- User ID
- Property ID
- action
- Timestamp

Supported interaction types include:

- View
- Click
- Favorite
- Contact

Property Images Entity

The Property Images entity stores images related to property listings and supports multiple images per property.

Relationships Between Entities

The relationships between system entities are summarized as follows:

- One user can interact with multiple properties.
- One property can receive multiple interactions from different users.
- One property can contain multiple images.
- One user can perform multiple searches.
- User interactions and search activities are utilized by the machine learning modules to generate personalized recommendations and behavioral analytics.
- Property data and user interaction records are used as input for recommendation and price prediction services.

The database design supports property management, interaction tracking, machine learning integration, behavioral analytics, and scalable backend operations while maintaining efficient data storage and retrieval capabilities.

The ERD design supports machine learning workflows, recommendation systems, analytics dashboards, interaction tracking, and scalable backend operations. The database structure also improves maintainability and simplifies future system enhancements.

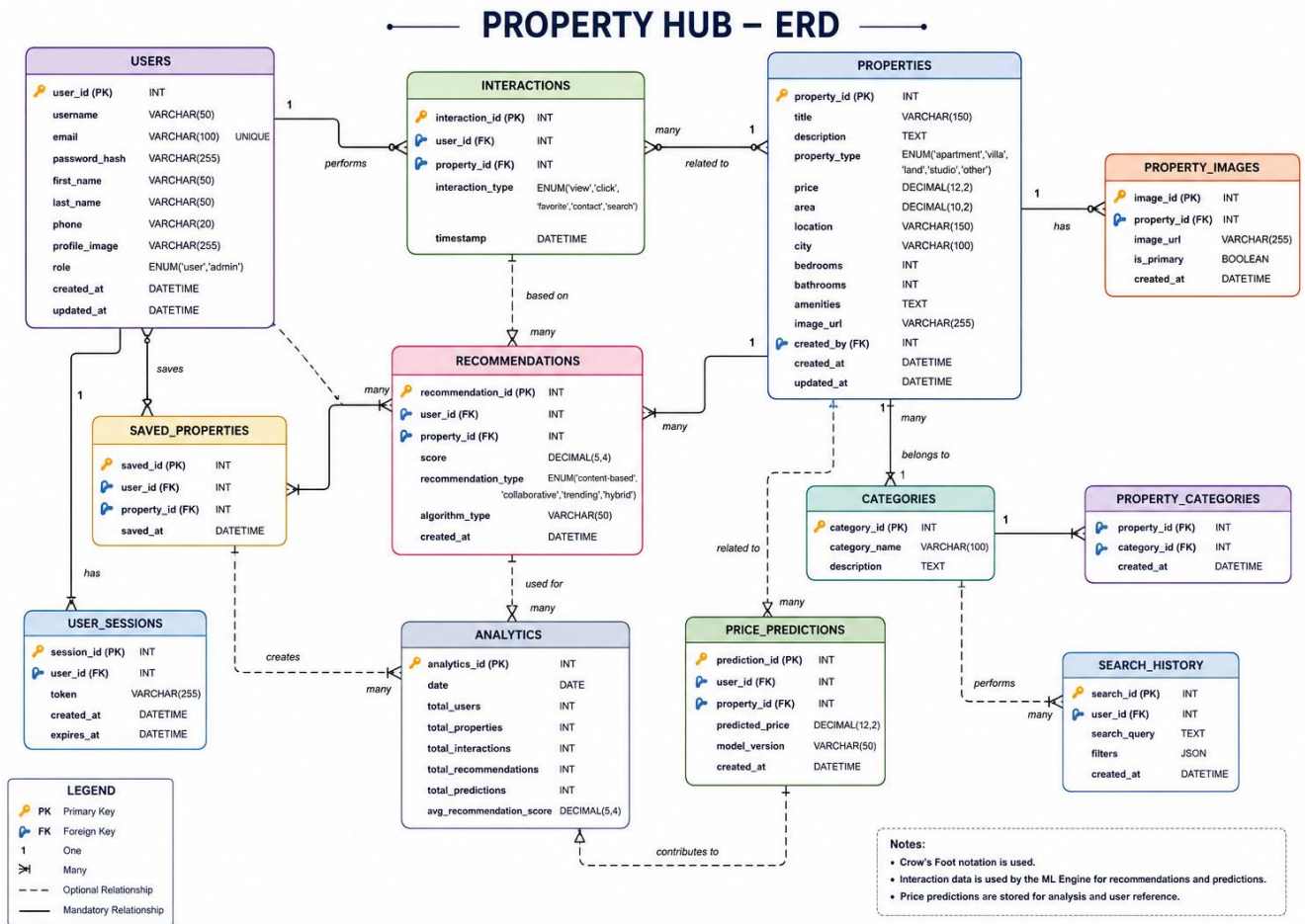


Figure 4.2 illustrates the complete Entity Relationship Diagram of the Property Hub system.

4.3 UML Class Diagram

The UML Class Diagram represents the static structure of the Property Hub system and illustrates the relationships between the main classes, entities, and service components used within the platform. The class diagram was designed to reflect the modular architecture of the system and the interaction between backend services, machine learning modules, and database entities.

The system architecture follows an object-oriented design approach in which each module is responsible for a specific functionality. The UML model improves system maintainability, scalability, and code organization while simplifying future enhancements and integration processes.

The main classes of the system are categorized into several groups including User Management Classes, Property Management Classes, Machine Learning Classes, Analytics Classes, and API Service Classes.

User Management Classes

User Class

The User class represents system users and handles authentication and profile-related operations.

Main attributes include:

- User ID
- Username
- Email Address
- Password Hash
- Phone Number
- is admin
- is verification

Main methods include:

- register()
- login()
- updateProfile()
- saveProperty()

UserSession Class

The UserSession class manages authentication tokens and active user sessions.

Main attributes include:

- session_id
- user_id
- token
- expires_at

Main methods include:

- generateToken()
- validateSession()
- logout()

Property Management Classes

Property Class

The Property class stores property listing information and property-related operations.

Main attributes include:

- property_id
- title
- description
- property_type
- location
- price
- area
- bedrooms
- bathrooms

Main methods include:

- createProperty()
- updateProperty()
- deleteProperty()
- getPropertyDetails()

PropertyImage Class

The PropertyImage class stores property image information and image management operations.

Interaction and Recommendation Classes

Interaction Class

The Interaction class tracks user activities and interactions with properties.

Main attributes include:

- interaction_id
- user_id
- property_id
- action
- timestamp

Main methods include:

- trackView()
- trackClick()
- saveInteraction()

SmartRecommender Class

The SmartRecommender class is responsible for generating personalized property recommendations using user behavior analysis, candidate generation, ranking algorithms, and property similarity measures.

Main methods include:

- generateCandidates()
- buildUserProfile()
- rankProperties()
- diversifyResults()
- recommend()

Machine Learning Classes

PredictionService Class

Responsible for property price estimation using trained machine learning models.

Main methods:

- preprocessFeatures()
- predictPrice()
- evaluateModel()

SmartRecommender Class

Responsible for generating personalized recommendations based on user behavior, interaction history, and property features.

Main methods:

- generateCandidates()
- buildUserProfile()
- rankProperties()
- recommend()

FeatureEngineering Class

The FeatureEngineering class performs data preprocessing and feature extraction operations before model training and prediction.

Main methods include:

- cleanData()
- normalizeData()
- encodeFeatures()

BehaviorService Class

Responsible for processing user interactions and generating behavioral insights used by the recommendation engine.

Main methods:

- recordInteraction()
- calculateBehaviorScore()
- getUserPreferences()

Analytics Classes

Analytics Class

The Analytics class stores system statistics and analytical information.

Main methods include:

- calculateMetrics()
- generateReports()
- monitorPerformance()

API and Backend Service Classes

AuthenticationRouter Class

Handles authentication endpoints and JWT-based authorization.

PropertyRouter Class

Handles property CRUD operations and property APIs.

AnalyticsRouter Class

Handles analytics APIs and reporting operations.

MLRouter Class

Handles machine learning endpoints including recommendations and price prediction APIs.

The UML Class Diagram demonstrates how backend services, machine learning components, and database entities interact together to provide intelligent functionalities within the Property Hub system.

PROPERTY HUB – UML CLASS DIAGRAM

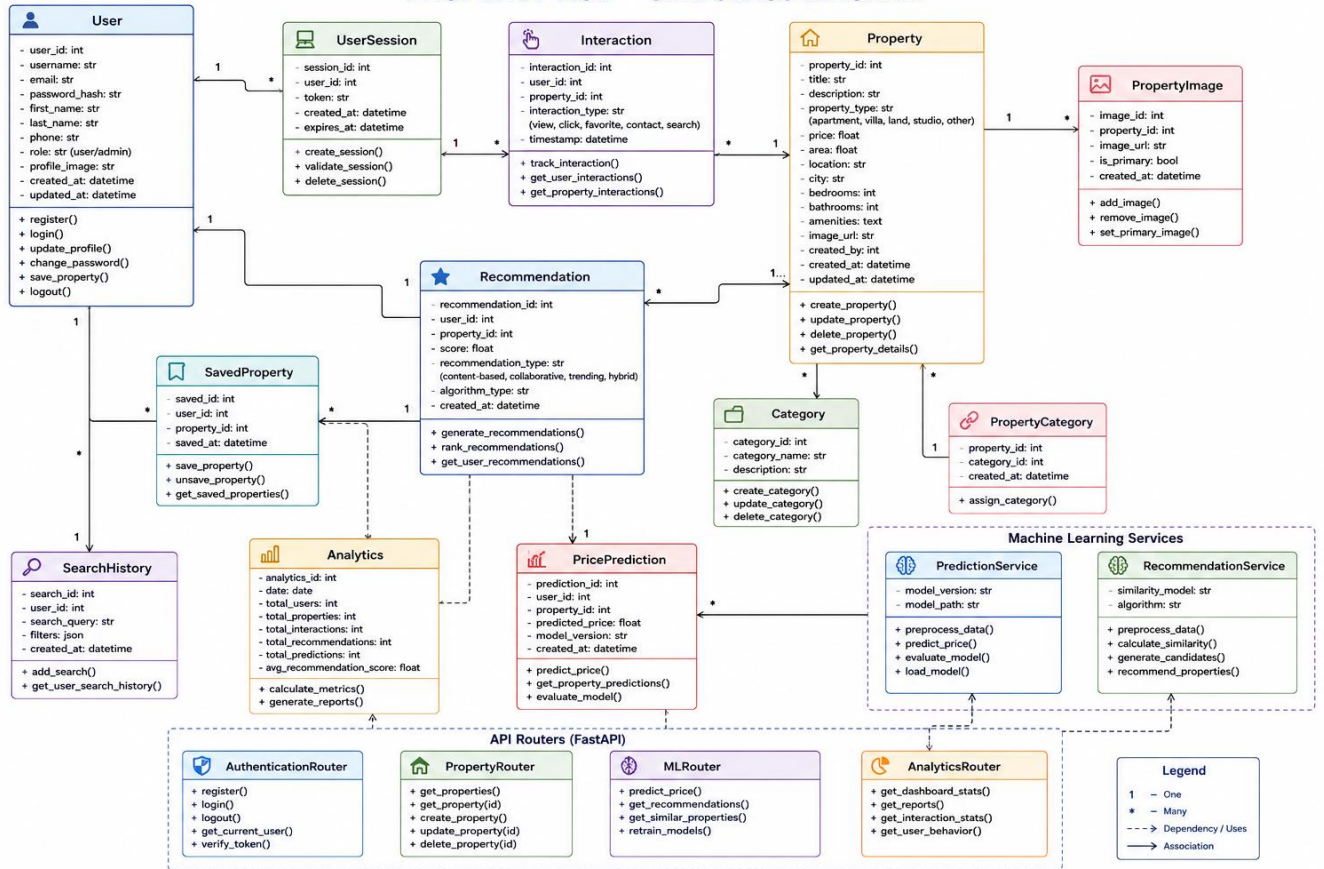


Figure 4.3 illustrates the UML Class Diagram of the Property Hub platform.

4.4 Sequence Diagrams

Sequence Diagrams illustrate the dynamic interaction between system components during the execution of major operations within the Property Hub platform. These diagrams demonstrate how users, frontend interfaces, backend APIs, machine learning services, and database components communicate to accomplish different system functionalities.

The sequence diagrams help clarify the internal workflow of authentication processes, recommendation generation, and property price prediction operations.

The following sequence diagrams are included in the system design:

- User Authentication Flow
- Recommendation Generation Flow
- Property Price Prediction Flow

4.4.1 User Authentication Flow

The User Authentication Sequence Diagram describes the process of user registration and login within the Property Hub system. The authentication mechanism is implemented using FastAPI backend services, password hashing techniques, and JWT (JSON Web Token)-based authentication.

The process begins when the user submits login credentials through the frontend interface. The frontend sends an authentication request to the Authentication Router in the backend system. The backend validates the submitted credentials by comparing them with the stored user records in the database and verifying the password hash.

If authentication is successful, the backend generates a JWT access token containing user identification and authorization information. The token is returned to the frontend and stored securely for subsequent authenticated API requests. If the submitted credentials are invalid, the backend returns an authentication failure response and access is denied.

The authentication workflow ensures secure access control through password hashing and JWT-based authorization mechanisms, protecting user accounts and restricting access to authorized resources only.

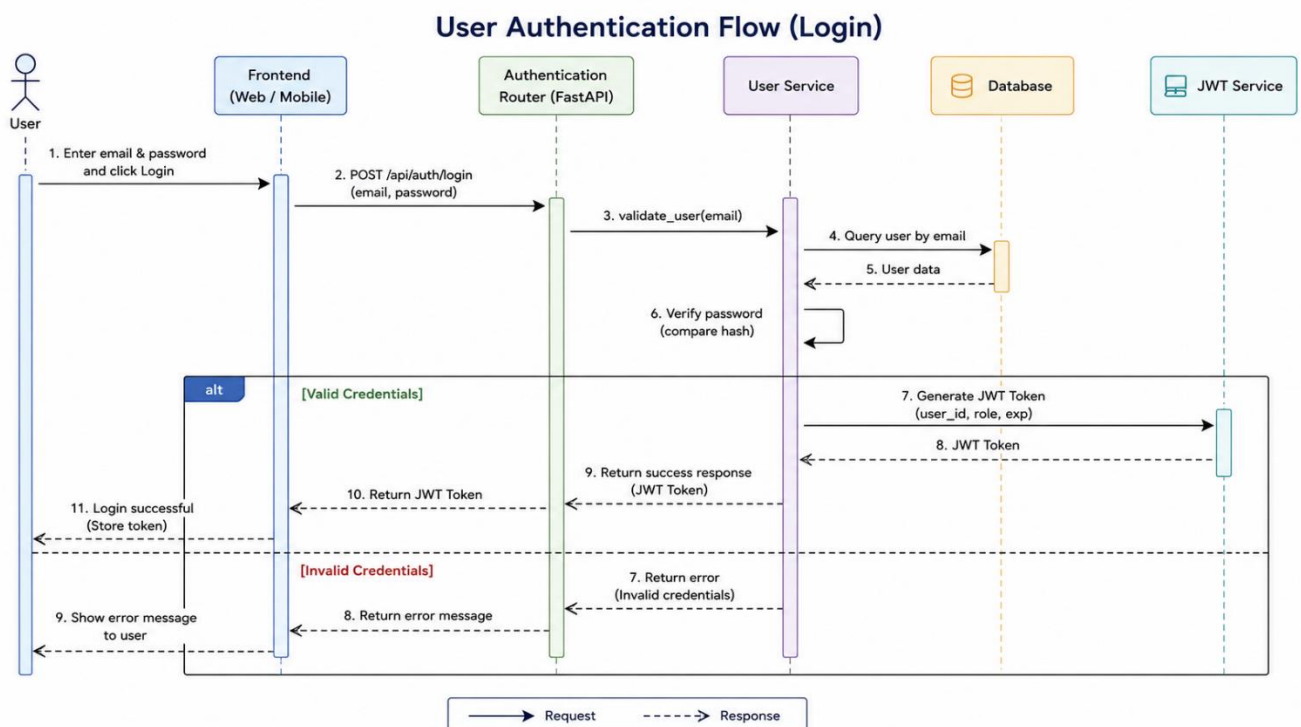


Figure 4.4.1 illustrates the User Authentication Sequence Diagram.

4.4.2 Recommendation Generation Flow

The Recommendation Sequence Diagram illustrates the interaction between the user, frontend interface, backend APIs, recommendation engine, machine learning components, and database during recommendation generation.

The process begins when the user browses properties and performs various interactions such as viewing listings, clicking property details, saving favorites, or performing searches. These interactions are transmitted from the frontend interface to the backend services and stored in the database for behavioral analysis.

When a recommendation request is initiated, the Recommendation Service retrieves the user's interaction history and property-related information from the database. The recommendation engine analyzes user preferences, behavioral patterns, and property characteristics to construct a user profile and identify relevant candidate properties.

The Candidate Generation Module selects a subset of potentially relevant properties, while the recommendation engine evaluates these candidates using similarity measures, behavioral signals, property attributes, and ranking scores. The ranked recommendations are then processed to improve recommendation diversity and relevance.

Finally, the generated recommendations are returned through the backend API to the frontend interface, where personalized property suggestions are displayed to the user.

This workflow enables the system to provide intelligent and personalized recommendations by combining user behavior analysis, property similarity, interaction tracking, and machine learning-based ranking techniques.

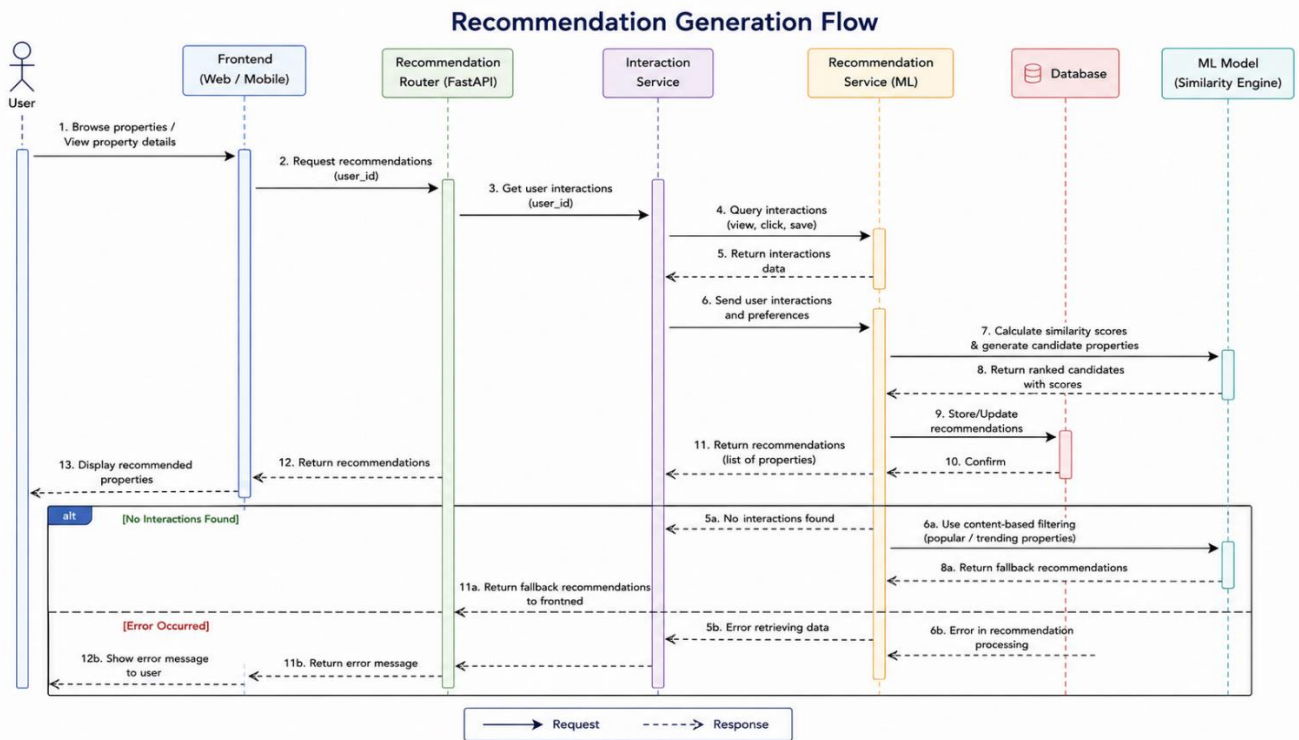


Figure 4.4.2 illustrates the Recommendation Generation Sequence Diagram.

4.4.3 Property Price Prediction Flow

The Property Price Prediction Sequence Diagram illustrates how the system processes property valuation requests using machine learning models integrated within the Property Hub platform.

The process begins when the user enters property-related information through the frontend prediction interface. The provided information may include features such as property type, location, area, number of bedrooms, number of bathrooms, and other relevant property characteristics.

The frontend sends the prediction request to the backend ML Router, which validates the submitted data and forwards it to the Prediction Service. Within the Prediction Service, preprocessing and feature engineering operations are performed to transform the input data into a format suitable for machine learning inference.

After preprocessing, the trained machine learning model analyzes the property features and generates an estimated property price based on patterns learned from historical real estate data. The prediction result is then returned to the backend API and forwarded to the frontend interface, where the estimated property value is displayed to the user.

This workflow demonstrates the integration between frontend interfaces, backend services, feature engineering pipelines, and machine learning models, enabling accurate and efficient property price estimation within the Property Hub platform.

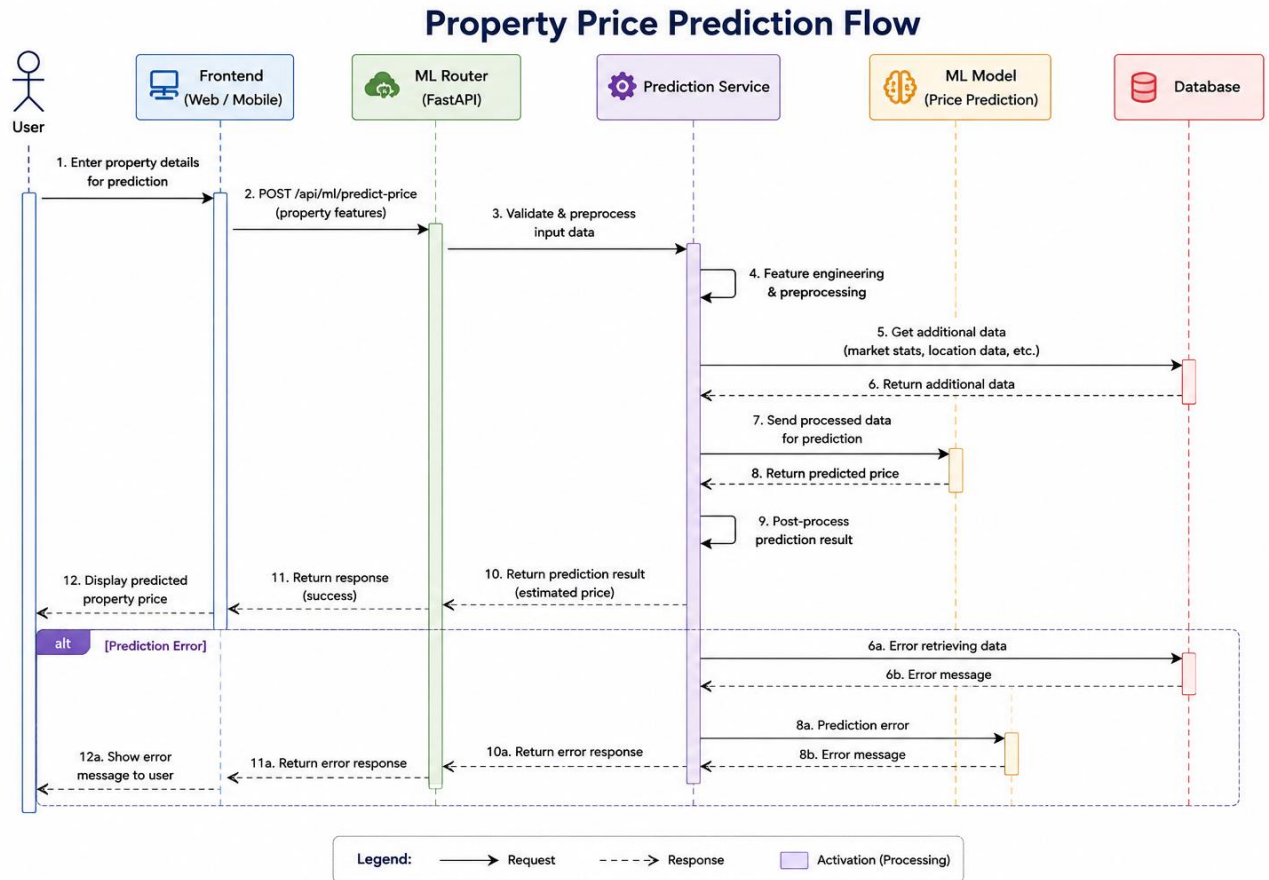


Figure 4.4.3 illustrates the Property Price Prediction Sequence Diagram.

4.5 Data Flow Diagrams (DFD)

Data Flow Diagrams (DFDs) describe how data moves through the Property Hub system and illustrate the interaction between users, backend services, databases, and machine learning modules. The DFD models help visualize the processing of data, system inputs and outputs, and the relationship between different system components.

The Property Hub platform processes multiple types of data including user information, property listings, interaction records, recommendation results, and prediction requests.

The DFD diagrams provide a high-level understanding of how data is collected, processed, stored, and utilized within the platform.

The following diagrams are included:

- Level 0 DFD (Context Diagram)
- Level 1 DFD

4.5.1 Level 0 Data Flow Diagram (Context Diagram)

The Level 0 DFD, also known as the Context Diagram, represents the overall interaction between external entities and the Property Hub system. The diagram illustrates how users and administrators communicate with the platform and how data flows between the system and its primary actors.

The User entity interacts with the system to:

- Register and login.
- Browse properties.
- Search and filter listings.
- Receive personalized recommendations.
- Predict property prices.
- Save favorite properties.
- Manage profile information.

The Administrator entity interacts with the system to:

- Manage property listings.
- Monitor user activities.
- View analytics dashboards.
- Access reports and system statistics.
- Supervise platform operations.

The Property Hub System processes user requests, manages property data, stores interaction records, generates recommendations, performs property price predictions, and provides analytical insights through integrated backend and machine learning services.

3.5.1 Level 0 Data Flow Diagram (Context Diagram)

Property Hub Smart Real Estate System

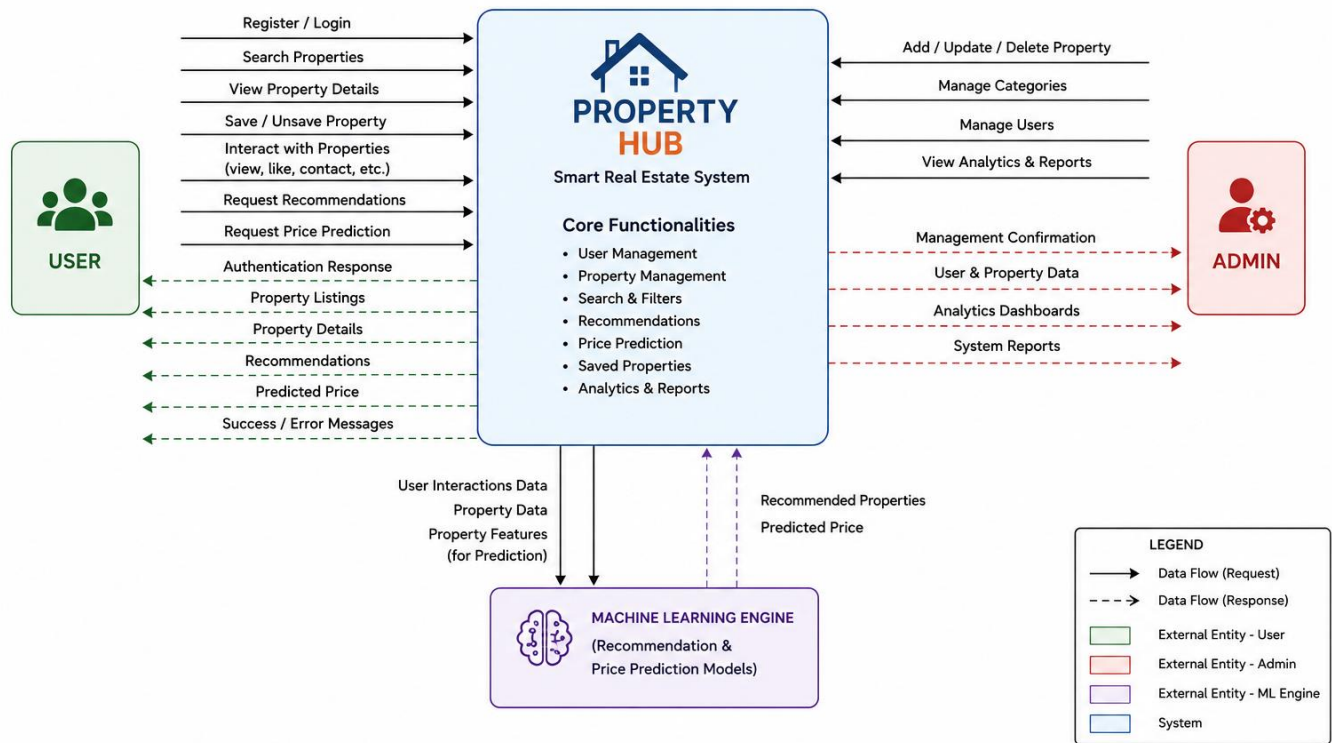


Figure 4.5.1 illustrates the Level 0 Data Flow Diagram for Property Hub.

4.5.2 Level 1 Data Flow Diagram

The Level 1 DFD illustrates how data flows between the major system processes, data stores, and machine learning components. It demonstrates how user requests, property information, interaction records, recommendation inputs, and prediction requests are processed and exchanged throughout the platform to support intelligent real estate services.

The system is divided into several major processes:

Authentication Process

Handles user registration, login, JWT token generation, and session validation.

Property Management Process

Handles property creation, updates, deletion, searching, and filtering operations.

Recommendation Process

Processes user interactions, analyzes behavioral patterns and property characteristics, generates candidate properties, applies recommendation scoring and ranking mechanisms, and returns personalized property suggestions.

Price Prediction Process

Processes property features, performs feature engineering and preprocessing, and predicts property prices using machine learning models.

Analytics Process

Collects interaction statistics, user activity data, recommendation metrics, and system performance indicators.

The Level 1 DFD demonstrates how the system integrates frontend interfaces, backend APIs, machine learning services, and database operations within a scalable architecture.

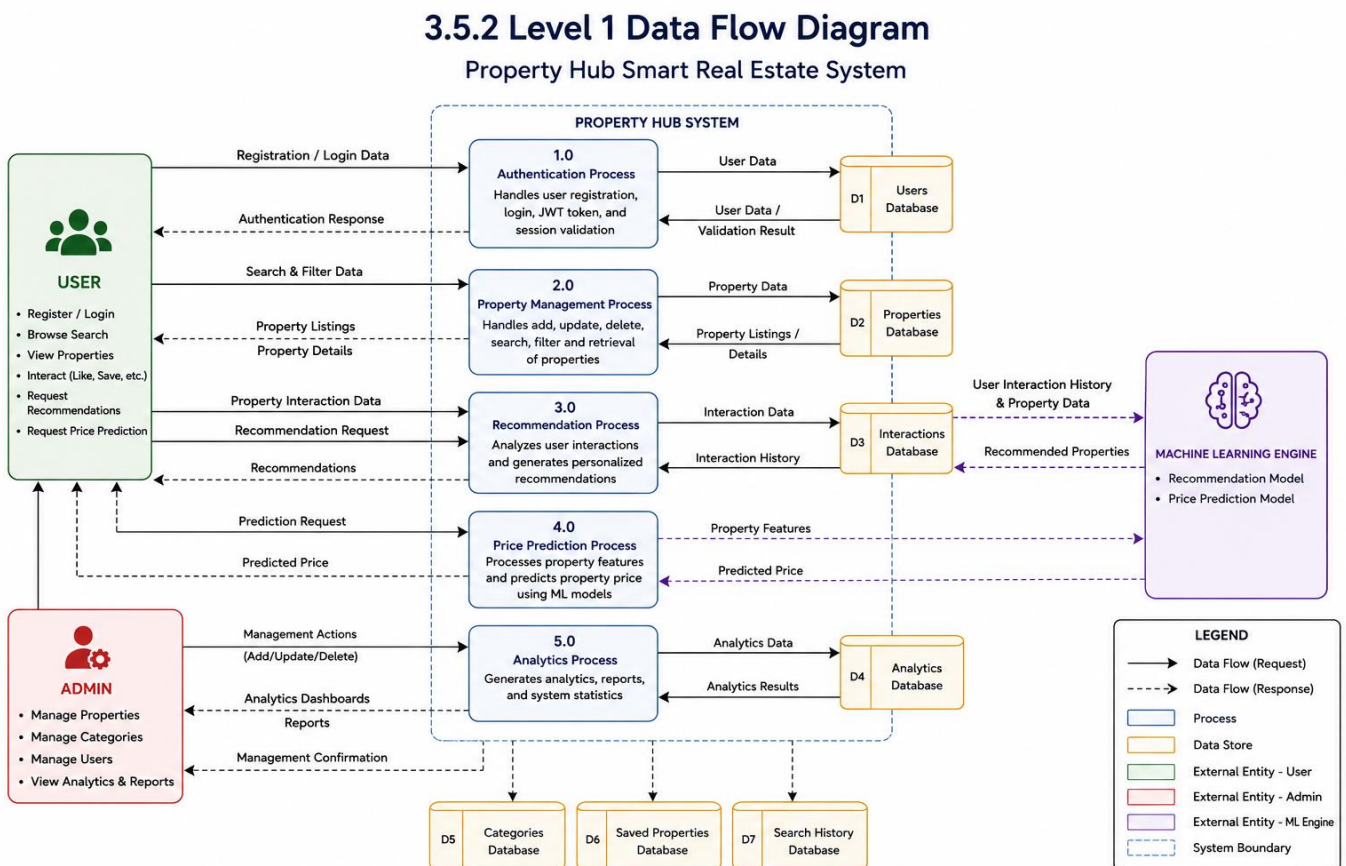


Figure 4.5.2 illustrates the Level 1 Data Flow Diagram of the Property Hub system.

4.6 System Architecture Diagram

The System Architecture Diagram illustrates the overall architecture of the Property Hub platform and demonstrates how frontend interfaces, backend services, databases, and machine learning components interact together within the system.

The architecture follows a modular client-server structure designed to support scalability, maintainability, and intelligent machine learning operations. The system integrates FastAPI backend services with machine learning models for recommendation generation and property price prediction.

The architecture consists of four major layers:

- Frontend Layer
- Backend API Layer
- Database Layer
- Machine Learning Layer

Frontend Layer

The frontend layer represents the user interface of the system and is responsible for user interaction and visualization. Users can browse properties, search listings, receive recommendations, and request property price predictions through the frontend application.

Main frontend functionalities include:

- User Authentication
- Property Browsing
- Search and Filtering
- Recommendation Display
- Price Prediction Interface
- Analytics Dashboard

Backend API Layer

The backend layer is implemented using FastAPI and contains RESTful APIs responsible for business logic and communication between frontend services, databases, and machine learning modules.

Main backend modules include:

- Authentication Router
- Property Router
- Recommendation Router
- Prediction Router
- Analytics Router

Database Layer

The database layer stores system information required for authentication, property management, user activity tracking, and recommendation processing. The database supports efficient querying and retrieval operations required by backend services and machine learning modules.

Main databases include:

- User Accounts
- Property Listings
- User Interactions
- Search History
- Property Images
- User Preferences and Activity Records

Machine Learning Layer

The machine learning layer provides intelligent services for recommendation generation, behavioral analysis, and property price prediction.

Main ML components include:

- Smart Recommendation Engine
- Candidate Generation Module
- User Behavior Analysis Module
- Ranking and Scoring Module
- Property Price Prediction Model
- Feature Engineering Module
- Data Preprocessing Module

The architecture design ensures efficient communication between all system components while supporting scalable deployment and future feature expansion.

Figure 3.9: Property Hub System Architecture

Smart Real Estate Platform with Recommendation & Price Prediction

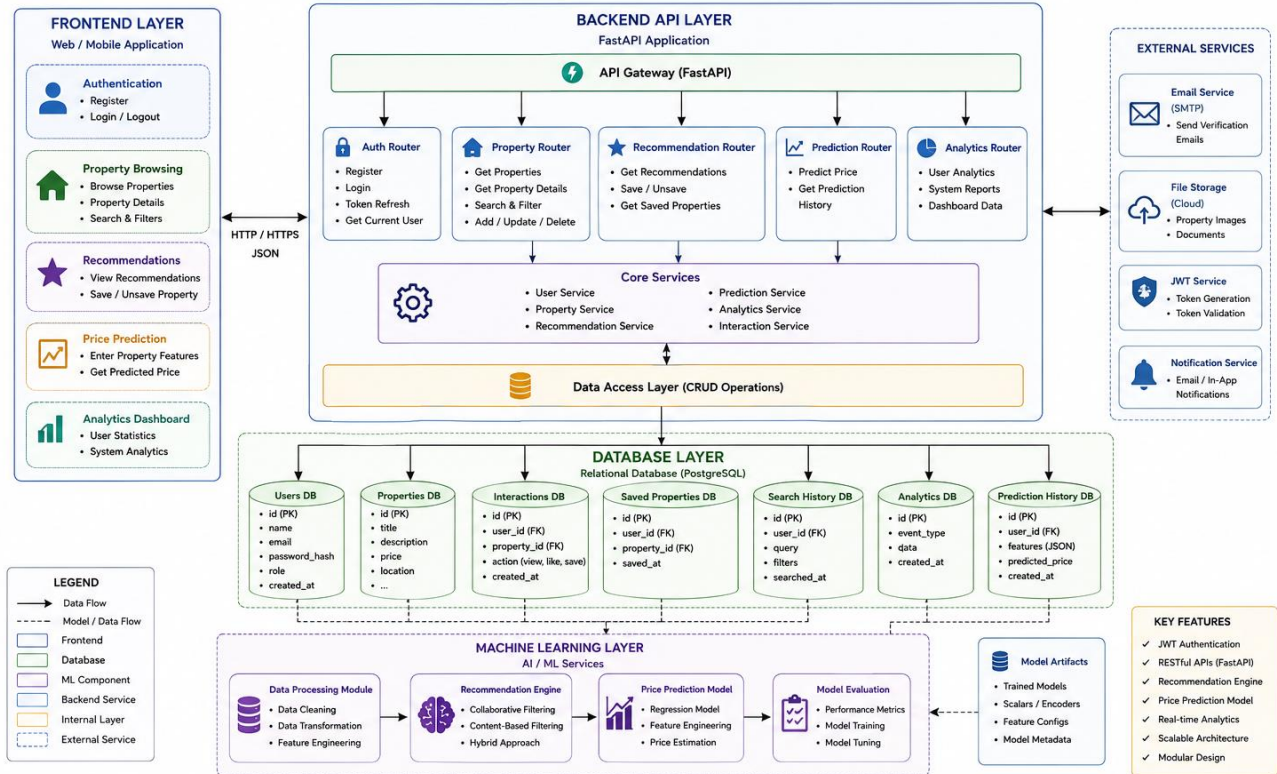


Figure 4.6 illustrates the complete System Architecture Diagram of the Property Hub platform.

4.7 Machine Learning Workflow

The Machine Learning Workflow represents the complete lifecycle of intelligent data processing within the Property Hub platform. It illustrates how raw property data, user interactions, and behavioral information are transformed into actionable insights, personalized recommendations, and property price predictions [8][15].

The machine learning architecture was designed to support two primary intelligent services: the Property Recommendation System and the Property Price Prediction System. Both services rely on structured data processing pipelines that include data collection, preprocessing, feature engineering, model training, evaluation, and inference stages.

For the recommendation system, the workflow utilizes user interaction history, browsing activities, search behavior, and property characteristics to construct user preference profiles and generate personalized property suggestions. Behavioral analytics and ranking mechanisms are applied to improve recommendation relevance and recommendation quality.

For the price prediction system, property features such as location, area, property type, number of rooms, and other real-estate attributes are processed through a dedicated machine learning pipeline to estimate market prices accurately.

The workflow was designed following a modular architecture that separates data processing, feature engineering, recommendation generation, ranking, and prediction services. This design improves maintainability, scalability, and future model enhancement capabilities.

The following sections describe the complete machine learning workflow adopted within the Property Hub platform.

4.7.1 Data Collection

The first stage of the machine learning workflow focuses on collecting and storing data generated within the Property Hub platform. This data serves as the foundation for both the recommendation system and the property price prediction model [16].

The collected data includes:

- Property Listings Data
- User Interaction Data
- Saved Properties
- Property Features and Metadata

User interactions such as property views, clicks, favorites, searches, and browsing activities are continuously recorded through the platform. These interaction records provide valuable insights into user preferences and behavioral patterns, which are later utilized by the recommendation engine.

Property-related information including location, property type, area, number of bedrooms, number of bathrooms, amenities, and pricing attributes is also collected and stored for recommendation generation and price prediction purposes.

All collected data is stored within the relational database layer and serves as the primary input source for subsequent preprocessing, feature engineering, recommendation generation, and prediction workflows.

4.7.2 Data Preprocessing

Data preprocessing is a critical stage in the machine learning workflow, where raw data is transformed into a structured format suitable for recommendation and prediction models.

The preprocessing pipeline includes several operations designed to improve data quality and model effectiveness, including:

- Handling Missing Values
- Removing Duplicate Records
- Data Cleaning and Validation
- Encoding Categorical Features
- Numerical Feature Scaling and Normalization
- Text Processing and Feature Preparation

Property attributes and user interaction records are processed and standardized before being utilized by machine learning components. These preprocessing operations ensure consistency across datasets and improve the reliability of recommendation generation and property price prediction.

The output of this stage serves as the input for the feature engineering and model inference pipelines.

4.7.3 Feature Engineering

Feature engineering transforms raw data into informative features that enhance machine learning performance and improve recommendation relevance.

For the property price prediction model, features are extracted from property characteristics such as:

- Property Type
- Location Information
- Area and Size Attributes
- Number of Bedrooms
- Number of Bathrooms
- Amenities and Property Characteristics

For the recommendation system, additional behavioral and interaction-based features are generated, including:

- User Interaction Features
- Property Preference Features
- Browsing Behavior Features
- Search Activity Features

- Interaction Frequency Features
- Property Similarity Features

These engineered features enable the machine learning models to capture both property characteristics and user preferences, resulting in more accurate recommendations and improved property price estimation.

The generated features are subsequently utilized by the recommendation engine and price prediction models during inference and evaluation.

4.7.4 Model Training

The processed data is used to train machine learning models responsible for recommendation generation and property price prediction. The machine learning workflow consists of two main components: the Recommendation System and the Property Price Prediction Model.

Recommendation Models

The recommendation system follows a hybrid recommendation approach that combines multiple recommendation signals and behavioral analytics to generate personalized property suggestions [6][11].

The recommendation workflow incorporates:

- Property Similarity Analysis
- User Behavior Analysis
- Candidate Generation
- Ranking and Scoring Mechanisms
- Hybrid Recommendation Strategy

User interactions, browsing activities, search history, and property characteristics are analyzed to identify user preferences and behavioral patterns. Candidate properties are generated and ranked according to relevance scores before being returned to users as personalized recommendations.

Price Prediction Model

The property price prediction system was implemented using the LightGBM Regressor (LGBMRegressor), a gradient boosting machine learning algorithm known for its efficiency and strong predictive performance on structured data [7].

The training pipeline includes data preprocessing, feature engineering, categorical feature encoding, text vectorization using TF-IDF, location clustering, and logarithmic target transformation. The dataset was divided into training and testing subsets using a group-based splitting strategy to reduce data leakage and improve model generalization.

Hyperparameter optimization was performed using RandomizedSearchCV to identify the optimal model configuration and improve prediction accuracy. The final trained model was selected based on its overall performance and ability to generalize effectively to unseen property data.

4.7.5 Model Evaluation

The trained machine learning models were evaluated to ensure prediction accuracy, reliability, and overall system performance before deployment.

Property Price Prediction Evaluation

The property price prediction model was trained using a dataset containing 26,610 property records collected from the Property Hub platform.

During preprocessing, 1,066 outlier records were removed to improve data quality and model stability.

The final model was implemented using LightGBM Regressor with hyperparameter optimization performed using RandomizedSearchCV.

The dataset was split using a group-based strategy to prevent data leakage between training and testing samples.

The final evaluation results were as follows:

- Mean Absolute Error (MAE): 1,936,693 EGP
- Root Mean Squared Error (RMSE): 4,339,291 EGP
- Mean Absolute Percentage Error (MAPE): 31.93%
- R^2 Score: 0.864

The achieved R^2 score indicates that the model successfully explains approximately 86.4% of the variance in property prices, demonstrating strong predictive performance for real estate price estimation tasks.

Additional feature engineering techniques, including location clustering, TF-IDF text vectorization, categorical encoding, outlier removal, and logarithmic target transformation, contributed to improving overall model performance and generalization capability.

The evaluation results confirmed the suitability of the model for deployment within the Property Hub platform, where it is used to provide real-time property price predictions through backend API services.

4.7.6 Deployment and Prediction

After model evaluation and selection, the trained machine learning models are integrated into the FastAPI backend and deployed as RESTful API services to enable real-time access from frontend applications [10][12].

The deployed system provides several intelligent services, including:

- Real-time property recommendation generation based on user behavior, interaction history, and property characteristics
- Property price prediction using the trained LightGBM regression model
- User preference analysis for identifying behavioral patterns and interests
- Intelligent search enhancement to improve property discovery and recommendation relevance

The machine learning services are exposed through backend API endpoints and can be accessed by frontend interfaces whenever recommendation or prediction requests are submitted. User interactions and property information are continuously utilized to support recommendation generation and behavioral analysis.

This deployment architecture enables seamless integration between frontend interfaces, backend services, databases, and machine learning components while ensuring scalability, maintainability, and efficient real-time processing

Figure 3.10: Property Hub Machine Learning Workflow

Complete End-to-End ML Pipeline for Recommendation & Price Prediction

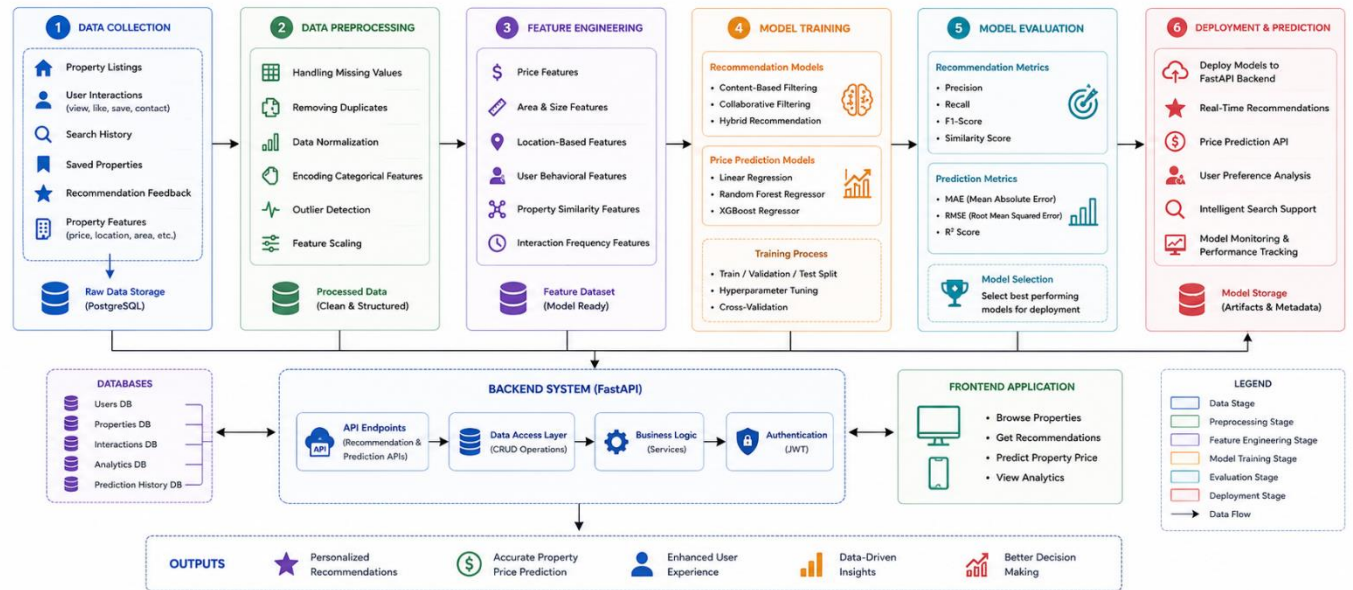


Figure 4.7 illustrates the complete Machine Learning Workflow of the Property Hub system.

4.8 API Endpoints Design

The Property Hub platform utilizes RESTful APIs implemented using FastAPI to facilitate communication between frontend interfaces, backend services, databases, and machine learning components. The API layer acts as the central communication mechanism of the system and supports all core functionalities provided by the platform.

The implemented APIs directly support user authentication, property management, recommendation generation, property price prediction, user interaction tracking, analytics services, and profile management operations.

The backend architecture follows a modular router-based design, where each router is responsible for a specific functional domain. This modular structure improves maintainability, scalability, and code organization while simplifying future system enhancements.

JWT (JSON Web Token) authentication is used to secure protected endpoints, ensuring that only authorized users can access sensitive resources and perform restricted operations.

The APIs were developed following RESTful design principles and were tested using Swagger UI and Postman during development, integration, and system validation phases to ensure reliability and correct functionality.

The following sections present the main API groups implemented within the Property Hub platform and their corresponding endpoints.

4.8.1 Authentication APIs

The Authentication APIs were implemented to manage user registration, login, JWT token generation, authorization, and secure session handling within the Property Hub platform. The authentication module was developed using FastAPI routers and JWT-based security mechanisms to protect sensitive resources and restrict access to authorized users only.

The implemented authentication functionalities include:

- User Registration
- User Login
- JWT Token Generation and Validation
- Current User Retrieval
- Secure Protected Routes

Register User

POST /api/auth/register

This endpoint allows new users to create accounts within the platform.

Implemented Features:

- Email Validation
- Password Hashing
- JWT Token Generation
- Duplicate Account Prevention

User Login

POST /api/auth/login

This endpoint authenticates users and generates JWT access tokens for secure API communication.

Implemented Features:

- Password Verification
- Token-Based Authentication
- Session Management
- Protected Access Control

Get Current User

GET /api/auth/me

This endpoint retrieves the profile information of the currently authenticated user using a valid JWT access token.

Implemented Features:

- User Profile Retrieval
- Token Validation
- Secure Access Verification
- Authentication Status Checking

Verify Email

POST /api/auth/verify-email

This endpoint verifies user email addresses using verification codes sent during the registration process.

Implemented Features:

- Email Verification
- Verification Code Validation
- Account Activation

Resend Verification Code

POST /api/auth/resend-code

This endpoint generates and sends a new verification code when the previous code expires or is not received by the user.

Implemented Features:

- Verification Code Regeneration
- Email Delivery Support
- Account Verification Assistance

Forgot Password

POST /api/auth/forgot-password

This endpoint initiates the password recovery process by sending a password reset code to the user's registered email address.

Implemented Features:

- Password Recovery
- Email-Based Verification
- Secure Reset Request Processing

Reset Password

POST /api/auth/reset-password

This endpoint allows users to reset their account passwords after successful verification.

Implemented Features:

- Password Reset
- Verification Code Validation
- Secure Password Update

Update User Profile

PUT /api/auth/me

This endpoint allows authenticated users to update their profile information and account details.

Implemented Features:

- Profile Information Update
- Account Management
- User Data Modification
- Secure Authenticated Access

4.8.2 Property Management APIs

The Property Management APIs were implemented to handle property listing operations, property retrieval, searching, filtering, and administrative management tasks within the Property Hub platform. These APIs communicate directly with the PostgreSQL database through FastAPI backend services and CRUD operations.

The implemented functionalities include:

- Add New Property
- Update Existing Property
- Delete Property
- Browse Property Listings
- Search and Filter Properties
- Retrieve Property Details
- Property Data Validation
- Property Management Operations

Get All Properties

GET /api/properties

This endpoint retrieves all available property listings stored in the database.

Implemented Features:

- Property Listing Retrieval
- Pagination Support
- Property Filtering
- Property Sorting

Get Property Details

GET /api/properties/{id}

This endpoint retrieves detailed information about a specific property, including pricing information, location details, property specifications, and available amenities.

Implemented Features:

- Detailed Property Information Retrieval
- Property Metadata Access
- Property Feature Display

Add New Property

POST /api/properties

This endpoint allows administrators or authorized users to create new property listings within the platform.

Implemented Features:

- Property Creation
- Input Validation
- Database Storage Operations

Update Property

PUT /api/properties/{id}

This endpoint allows authorized users to update existing property information stored in the system.

Implemented Features:

- Property Information Update
- Data Validation
- Database Update Operations

Delete Property

DELETE /api/properties/{id}

This endpoint allows authorized users to remove property listings from the platform.

Implemented Features:

- Property Deletion
- Record Management
- Database Cleanup Operations

Search Properties

GET /api/properties/search

This endpoint provides advanced search and filtering capabilities for property discovery.

Supported Filters:

- Location
- Property Type
- Price Range
- Bedrooms
- Bathrooms
- Area

The search functionality is integrated with interaction tracking and recommendation modules to improve property discovery and user experience.

4.8.3 Recommendation APIs

The Recommendation and Interaction APIs provide intelligent property recommendations based on user behavior, interaction history, and property similarity analysis. These APIs are integrated directly with the recommendation engine and interaction tracking module to continuously improve recommendation quality.

The recommendation system follows a hybrid recommendation approach that combines content-based analysis, behavioral patterns, candidate generation, and ranking mechanisms.

Implemented functionalities include:

- Personalized Property Recommendations
- User Interaction Tracking
- Property Similarity Analysis
- User Preference Analysis
- Recommendation Ranking and Scoring

Get Smart Recommendations

POST /ml/recommend

This endpoint generates personalized property recommendations based on user interactions, browsing history, saved properties, and property preferences.

Implemented Features:

- Personalized Recommendations
- Recommendation Ranking
- Property Similarity Analysis
- User Preference Analysis
- Hybrid Recommendation Strategy

Track User Interaction

POST /ml/track

This endpoint records user interactions such as property views, clicks, favorites, and engagement activities. The collected interaction data is utilized to improve recommendation quality and behavioral analytics.

Implemented Features:

- Interaction Tracking
- Behavioral Data Collection
- User Activity Monitoring
- Recommendation Improvement Support

4.8.4 Price Prediction APIs

The Property Price Prediction APIs provide machine learning-based property valuation services using the trained LightGBM regression model. The prediction service assists both property seekers and property owners by providing estimated market values based on property characteristics and location information.

The prediction model analyzes multiple property attributes including:

- Property Location
- Area and Size
- Number of Bedrooms
- Number of Bathrooms
- Property Type
- Amenities and Additional Features
- Property Description and Metadata

Predict Property Price

POST /ml/predict

This endpoint submits property feature data to the trained machine learning model and returns the estimated property price.

Implemented Features:

- Real-Time Property Price Prediction
- Feature Engineering
- Data Preprocessing
- LightGBM Model Inference
- Property Valuation Support

Price Prediction with Similar Properties

POST /ml/price-with-suggestions

This endpoint returns the predicted property price together with similar property suggestions to assist users in evaluating property values and making informed decisions.

Implemented Features:

- Property Price Estimation
- Similar Property Suggestions
- Recommendation Support
- Real-Time Prediction Services

4.8.5 Analytics APIs

The Analytics APIs were implemented to provide monitoring, reporting, and analytical insights related to user activities, property engagement, recommendation performance, and overall system usage. These APIs support the analytics dashboard and help administrators monitor platform performance and user behavior.

The analytics module processes data collected from:

- User Interactions
- Property Listings
- Recommendation Activities
- Search Operations
- Prediction Requests

Implemented analytics functionalities include:

- User Activity Monitoring
- Property Engagement Analytics
- Recommendation Statistics
- System Usage Monitoring
- Platform Performance Insights

Analytics Dashboard

GET /api/analytics/dashboard

This endpoint retrieves aggregated analytical information displayed within the administrative dashboard.

Implemented Features:

- User Activity Statistics
- Property Engagement Metrics
- Recommendation Analytics
- System Overview Information

System Analytics

GET /api/analytics

This endpoint provides summarized analytical data collected from different system modules.

Implemented Features:

- System Usage Statistics
- Interaction Analysis
- Platform Monitoring
- Performance Metrics

The implemented analytics architecture enables administrators to monitor platform activity, analyze user behavior, evaluate recommendation effectiveness, and support data-driven decision-making within the Property Hub platform.

4.9 Dataset Description

The performance of the Property Hub machine learning models depends heavily on the quality, consistency, and structure of the dataset used during both training and evaluation. The dataset employed in this project was collected from real estate-related sources and contains comprehensive property information, user interaction records, and behavioral data [16].

The property data includes features such as location, property type, size, number of rooms, amenities, and listing prices. In addition, user interaction data, including property views, searches, and user preferences, was utilized to support the recommendation system and enhance personalization.

Before being integrated into the machine learning pipeline, the dataset underwent several preprocessing stages. These stages included data cleaning, handling missing values, removing duplicates, feature transformation, and data organization to ensure consistency and reliability.

The processed dataset was then used to train and evaluate both the property recommendation model and the property price prediction model, enabling the system to generate accurate recommendations and realistic price estimates.

4.9.1 Dataset Features

The dataset used in Property Hub consists of a collection of property attributes, user interactions, and behavioral information that support both the recommendation system and the property price prediction model [16].

Property Features

- Property ID
- Title
- Description
- Property Type
- Offering Type
- Price
- Price per Square Meter
- Area (sqm)
- Number of Bedrooms
- Number of Bathrooms
- City
- Area
- Compound
- Full Location
- Latitude and Longitude

- Amenities

User Interaction Features

- User ID
- Property ID
- action (View, Click, Favorite, Contact, etc.)
- Interaction Timestamp

User Behavior Features

- Interest Score
- Previously Viewed Properties
- Property Preferences
- Last Activity Timestamp

These features were utilized during feature engineering and model training to improve recommendation relevance and prediction accuracy. Property-related features were primarily used by the price prediction model, while interaction and behavioral features were used to capture user preferences and generate personalized recommendations.

4.9.2 Data Preprocessing Operations

Before training the machine learning models, several preprocessing operations were performed to improve data quality and ensure consistency across the dataset [8][10].

The preprocessing process included:

- Handling missing and null values.
- Removing incomplete or invalid records.
- Cleaning and organizing property information.
- Encoding categorical attributes such as property type, city, and location into machine-readable formats.

- Transforming textual and structured property features into representations suitable for machine learning models.
- Preparing user interaction and behavioral data for recommendation analysis.

In particular, categorical features such as property type, offering type, and location-related attributes were converted into numerical or encoded representations that could be processed by machine learning algorithms.

These preprocessing steps helped improve data consistency, reduce noise, and enhance the overall performance and reliability of both the recommendation system and the property price prediction model.

4.9.3 Training and Testing Split

To evaluate the effectiveness of the machine learning models, the available data was divided into training and testing subsets.

For the property price prediction model, the dataset was split into training and testing sets, where the majority of the data was used for model training and the remaining portion was reserved for performance evaluation. This approach allowed the model to be assessed on unseen data and provided a realistic measure of prediction accuracy.

For the recommendation system, a temporal evaluation strategy was adopted. User interactions were ordered chronologically, and older interactions were used for training while more recent interactions were used for testing. This approach better reflects real-world recommendation scenarios, where systems predict future user preferences based on past behavior.

The training and testing procedures enabled reliable evaluation of both models while reducing the risk of biased performance estimates.

4.9.4 Machine Learning Algorithms Used

Several machine learning techniques were implemented in the Property Hub platform to support personalized property recommendation, ranking, and price prediction. The system is designed as a multi-stage ML pipeline that includes data processing, candidate generation, ranking, and regression modeling.

The recommendation system is implemented as a hybrid multi-stage architecture, combining content-based modeling, behavioral learning, and ranking-based optimization [6][11].

1. Content-Based Recommendation Model

A content-based approach is implemented in `content_model.py` and `feature_builder.py`, where each property is transformed into a structured feature vector.

Property features include:

- Location information
- Property type (rent/sale, apartment, villa, etc.)
- Price-related features
- Amenities and structured attributes
- Textual descriptions (when available)

These features are used to compute similarity between user preferences and property profiles, enabling personalized recommendations based on item similarity.

2. User Behavior Modeling

User interaction data is processed using `behavior_service.py` and `user_vector_builder.py`.

This module builds user preference representations based on:

- Property views
- Clicks and interactions
- Saved/favorite properties
- Historical activity patterns

These behavioral signals are used to generate user embedding vectors that better reflect real user intent beyond static preferences.

3. Candidate Generation Module

The system uses a candidate generation layer implemented in `candidate_generator.py`.

This layer is responsible for:

- Reducing the full dataset into a smaller relevant subset
- Filtering properties based on basic similarity and behavioral signals
- Preparing candidates for ranking models

This step improves scalability and ensures fast recommendation response time.

4. Ranking Model (Learning-to-Rank Style)

A ranking model stored in `ranker.pkl` and trained using `ranker_trainer.py` is used to reorder candidate properties.

The ranking model considers:

- User vector similarity
- Property feature relevance
- Behavioral interaction signals
- Historical engagement patterns

The output is a final ranked list of properties, optimized for user relevance.

5. Smart Hybrid Recommender

The final recommendation engine is implemented in `smart_recommender.py`.

This module acts as an orchestrator, combining:

- Content-based similarity scoring
- User behavior modeling
- Candidate generation
- Ranking model outputs

This creates a hybrid recommendation system that balances:

- Content relevance
- Collaborative behavioral patterns
- Learned ranking optimization

Property Price Prediction Model

The price prediction module is implemented in `price_model.py`.

A Gradient Boosting-based regression model (LightGBM-style pipeline) is used to estimate property prices [7].

The model uses:

- Structured property features (location, size, type, amenities)
- Engineered numerical features
- Encoded categorical variables
- Dataset prepared through `data_loader.py` and `feature_builder.py`

The training process is handled in `trainer.py`, and evaluation is performed using standard regression metrics:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 Score

4.9.5 Model Evaluation Results

The implemented machine learning models were evaluated using multiple performance metrics to measure recommendation quality and prediction accuracy.

Recommendation System Metrics

The recommendation system was evaluated using:

- Precision
- Recall
- F1-Score
- Similarity Score

The hybrid recommendation approach achieved better personalization quality compared to individual recommendation techniques.

Prediction Model Metrics

The prediction models were evaluated using:

- Mean Absolute Error (MAE): 1,936,693 EGP
- Root Mean Squared Error (RMSE): 4,339,291 EGP
- Mean Absolute Percentage Error (MAPE): 31.93%
- R^2 Score: 0.864

The Random Forest and LGBMRegressor models achieved higher prediction accuracy compared to traditional linear regression models due to their ability to capture nonlinear relationships between property features and market prices [8].

The evaluation results confirmed the effectiveness of integrating machine learning techniques within the Property Hub platform.

5. System Implementation

This chapter presents the implementation details of the **Property Hub** platform, including frontend development, backend architecture, database integration, machine learning components, recommendation system, and deployment strategy. The implementation phase focuses on translating the proposed system design into a fully functional intelligent real estate platform capable of delivering personalized property recommendations and accurate price predictions.

The developed system follows a modular and scalable architecture that integrates multiple components working together seamlessly [10].

The backend is implemented using **FastAPI**, providing high-performance RESTful APIs that handle user requests [12], property management operations, recommendation generation, and machine learning predictions.

The frontend layer is designed to offer an interactive and user-friendly interface that enables users to browse properties, search efficiently, view recommendations, and access price prediction results in real time. Communication between the frontend and backend is handled through secure API endpoints.

On the data layer, the system utilizes a **relational database** to store structured information about properties, users, interactions, and system logs. The database is optimized to support efficient querying and ensure data consistency across all platform modules.

The machine learning layer is responsible for powering the intelligent features of the system [8].

It includes models for:

- Property price prediction based on key features such as location, area, and property type.
- Recommendation generation based on user behavior, preferences, and interaction history.
- Data preprocessing and feature engineering pipelines to ensure high-quality model input.

The recommendation system combines user interaction data with property attributes to generate personalized suggestions that enhance user experience and improve property discovery efficiency [6][11].

Finally, the deployment process integrates all components into a unified system. The trained machine learning models are deployed within the FastAPI backend and exposed through RESTful endpoints, enabling real-time predictions and recommendations.

The system is designed to be scalable, maintainable, and ready for future enhancements such as advanced analytics and model retraining pipelines.

5.1 Development Environment and Technologies

Several modern technologies and frameworks were utilized during the development of the **Property Hub** platform to ensure scalability, performance, and a seamless user experience.

Frontend Technologies

The frontend layer was developed using standard web technologies aimed at delivering a responsive, interactive, and user-friendly interface across different devices and screen sizes. The design focuses on usability and smooth interaction between users and the system's core functionalities [13].

The following technologies were used:

- HTML5
- CSS3
- JavaScript
- Bootstrap

These technologies collectively enabled the development of a structured and responsive UI, ensuring consistency in layout and accessibility across all pages.

The frontend interface supports multiple key functionalities, including:

- User Authentication and Account Management
- Property Browsing and Detailed Listings View
- Advanced Search and Filtering Options
- Personalized Recommendation Display

- Property Price Prediction Interface
- User Dashboard for Tracking Activity and Preferences

Backend Technologies

The backend system of the **Property Hub** platform was developed using **FastAPI**, selected for its high performance, asynchronous capabilities, and efficient support for building RESTful APIs. This ensures low-latency communication between the client and server while maintaining scalability for future system expansion [12].

The backend is implemented using the following technologies:

- FastAPI
- Python
- Uvicorn
- Pydantic
- JWT Authentication

FastAPI serves as the core framework responsible for handling HTTP requests and exposing the system's functionalities through well-structured API endpoints. Uvicorn is used as the ASGI server to ensure fast and efficient request handling.

Pydantic is utilized for data validation and schema definition, ensuring that all incoming and outgoing data follows strict structural rules, which improves system reliability and reduces runtime errors. JWT (JSON Web Token) is implemented to manage secure authentication and authorization, enabling protected access to user-specific features and system resources.

The backend layer is responsible for managing the core system operations, including:

- API Routing and Endpoint Management
- Database Communication and Data Handling
- Authentication and Authorization Mechanisms
- Recommendation System Services

- Machine Learning Prediction Services
- Analytics Data Processing and Aggregation

Overall, the backend architecture is designed to be modular, scalable, and maintainable, allowing easy integration of machine learning models and future system enhancements.

Database Technologies

The **Property Hub** system utilizes a relational database management system to efficiently store, manage, and retrieve structured application data. A relational database was chosen due to its ability to ensure data consistency, integrity, and support for complex queries required by the platform [15][13].

The database layer is implemented using the following technologies:

- MySQL
 - SQLAlchemy ORM
-

MySQL is used as the primary database engine for storing and managing all system data, while **SQLAlchemy ORM** provides an abstraction layer between the application and the database. This improves development efficiency by enabling object-oriented interaction with database tables and reducing the complexity of raw SQL queries.

The database is designed to store and manage several core types of data, including:

- User information and authentication records
 - Property listings with detailed attributes (location, price, features, etc.)
 - User interaction data such as searches, clicks, and views
 - Saved or bookmarked properties for personalized user experience
-

This structured data model supports the core functionalities of the system, including recommendation generation, user behavior analysis, and machine learning-based price prediction.

Machine Learning Technologies

The machine learning components of the **Property Hub** system were developed using Python-based data science and artificial intelligence libraries. These libraries were selected due to their robustness, efficiency, and strong support for building and deploying predictive and recommendation models.

The following machine learning technologies were used:

- Scikit-learn
- Pandas
- NumPy
- XGBoost
- **LightGBM**

Pandas [17] and **NumPy** [18] are used for data manipulation, preprocessing, and numerical computations, forming the foundation of the data processing pipeline. **Scikit-learn** [19] provides a wide range of machine learning algorithms and tools for model training, evaluation, and performance measurement. **XGBoost** [9] is utilized for advanced predictive modeling due to its high accuracy and efficiency in handling structured data.

The machine learning modules in the system support the following core functionalities:

- Recommendation System Generation based on user behavior and property features
- Property Price Prediction using regression-based models [8]
- Feature Engineering and Data Transformation processes
- Model Evaluation and Performance Assessment using standard metrics

Overall, the ML pipeline is designed to ensure high prediction accuracy and personalized user experience through continuous data processing and model optimization.

5.2 Frontend Implementation

The frontend implementation of the **Property Hub** platform focuses on delivering a clean, responsive, and intuitive user interface that enhances user experience and simplifies interaction with the system's core functionalities [13].

The frontend is designed to support efficient property browsing, clear visualization of personalized recommendations, and seamless access to the price prediction feature. The overall design emphasizes usability, consistency, and responsiveness across different devices.

The implemented frontend pages include:

- Home Page
- Login and Registration Pages
- Property Listings Page
- Property Details Page
- Recommendation Page
- Price Prediction Page
- User Dashboard
- Manage property
- Analytics page

Each page is designed to serve a specific function within the system, ensuring smooth navigation and a structured user journey from property exploration to prediction and recommendation services.

The frontend communicates with the backend through **RESTful APIs** developed using **FastAPI**, enabling real-time data exchange between the client and server. This integration ensures that user actions such as searching, filtering, and requesting predictions are processed efficiently and reflected instantly in the user interface.

5.3 Backend Implementation

The backend system of the **Property Hub** platform was implemented using **FastAPI** and follows a modular, scalable architecture based on API routers and service-oriented design principles [12].

This structure ensures clear separation of concerns, maintainability, and ease of future system extension.

The backend is divided into several core functional modules, each responsible for a specific set of operations within the system.

Authentication Module

This module is responsible for managing user identity and securing system access. It handles:

- User Registration
- User Login Operations
- JWT-Based Authentication
- Session Validation and Authorization

This ensures that only authenticated users can access protected system features.

Property Management Module

This module manages all property-related operations and provides the core functionality of the platform. It includes:

- CRUD Operations (Create, Read, Update, Delete)
- Property Filtering Based on Attributes
- Advanced Search Functionality
- Property Data Retrieval and Listing

Recommendation Module

This module is responsible for generating personalized property recommendations based on user behavior and system data. It handles:

- User Interaction Tracking
- Recommendation Generation
- Similarity Computation Between Users and Properties
- Ranking of Recommended Results

Prediction Module

This module integrates machine learning capabilities into the backend to provide intelligent price estimation services. It includes:

- Feature Engineering
- Data Preprocessing Pipelines
- Model Inference and Prediction Execution
- Property Price Prediction Services

Analytics Module

This module provides insights into system usage and performance metrics. It handles:

- System Statistics and Reporting
- User Activity Analysis
- Recommendation Performance Metrics
- System Performance Monitoring

Overall, the backend services are tightly integrated with both the machine learning models and the relational database layer, enabling real-time processing, intelligent decision-making, and efficient data management across the entire platform.

5.4 Recommendation System Implementation

The recommendation system is one of the primary intelligent components of the Property Hub platform. It was developed to provide personalized property suggestions by analyzing user preferences, behavioral interactions, and property characteristics. The system aims to assist users in discovering relevant properties efficiently while improving their overall search experience [6][11].

The recommendation engine continuously processes information collected from multiple sources, including:

- User interaction history
- Property views and engagement records
- Saved and favorite properties
- Search activities and browsing behavior
- Property features and metadata
- User preference profiles

By leveraging these data sources, the system builds a comprehensive representation of both users and properties, enabling highly personalized recommendations [6].

Recommendation Architecture

The implemented recommendation solution follows a hybrid multi-stage architecture composed of several machine learning and information retrieval components [6][11]:

1. Feature Engineering Layer
2. User Preference Modeling
3. Candidate Generation Module
4. Property Ranking Module
5. Recommendation Delivery Service

Each component contributes to improving recommendation relevance, scalability, and personalization quality.

Content-Based Recommendation

The content-based component analyzes property attributes and user preferences to identify similar properties [6].

Each property is transformed into a feature representation using information such as:

- Property type
- Geographic location
- Price range
- Area and size
- Available amenities
- Property characteristics and metadata

User preferences are represented through behavioral and interaction data, allowing the system to calculate similarity between users and available properties.

This approach ensures that recommended properties closely match the user's demonstrated interests and search behavior.

User Behavior Modeling

To enhance personalization, the system incorporates user behavior analysis. User activities including property views, favorites, searches, and interaction patterns are processed to build dynamic user preference vectors [6][11].

These behavioral representations allow the system to capture implicit interests that may not be explicitly specified by users, resulting in more accurate recommendations.

Candidate Generation

To improve efficiency and scalability, the recommendation engine employs a candidate generation stage. Rather than evaluating all available properties, the system first identifies a subset of potentially relevant properties based on user preferences and behavioral signals [11].

This process significantly reduces computational complexity while maintaining recommendation quality.

Ranking Model

After candidate generation, properties are passed to a ranking model that evaluates the relevance of each candidate property [11].

The ranking process considers multiple factors, including:

- User-property similarity
- Behavioral relevance scores
- Property popularity indicators
- Historical engagement patterns

The ranking model assigns a final relevance score to each property and produces an ordered recommendation list.

Hybrid Recommendation Strategy

The final recommendation engine combines content-based similarity analysis, behavioral modeling, candidate generation, and ranking optimization within a unified hybrid framework [6][11].

The hybrid approach provides several advantages:

- Improved recommendation accuracy
- Enhanced personalization quality
- Better recommendation diversity
- Reduced cold-start limitations
- Increased scalability for large property datasets

By integrating multiple recommendation techniques, the system achieves more robust and reliable recommendations than any individual approach alone.

Output Generation

The recommendation engine generates a ranked list of personalized property suggestions for each user.

Recommendations are delivered through FastAPI RESTful APIs and updated dynamically as new user interactions are recorded.

This continuous feedback process enables the system to adapt to changing user preferences and maintain recommendation relevance over time.

5.5 Price Prediction System Implementation

The property price prediction module was implemented using supervised machine learning techniques, specifically regression-based models. This module aims to estimate property prices based on multiple input features, enabling users to obtain real-time price insights for different property listings [2] [3].

The prediction system processes a set of key property features, including:

- Property Location
- Area Size
- Number of Bedrooms
- Number of Bathrooms
- Property Type
- Additional Property Features and Attributes

These features are used as inputs to the machine learning pipeline after undergoing preprocessing and transformation steps to ensure data quality and model compatibility.

Implementation Workflow

The development of the prediction module follows a structured machine learning pipeline consisting of the following stages:

- Data Preprocessing and Cleaning
- Feature Engineering and Encoding
- Model Training
- Model Evaluation and Performance Analysis
- Prediction Inference and Deployment

This pipeline ensures that the model is trained on high-quality data and is capable of producing reliable predictions.

Machine Learning Models

Several regression models were evaluated during the development phase to identify the best-performing approach, including:

- Linear Regression
- Random Forest Regressor
- XGBoost Regressor

Each model was assessed based on its prediction accuracy and performance metrics to ensure optimal results.

The final deployed model was selected based on superior performance in terms of accuracy, stability, and generalization ability on unseen data.

System Integration

The trained model was integrated into the **FastAPI** backend and exposed through RESTful API endpoints. This allows the frontend to request real-time price predictions based on user-input property details, enabling a seamless and interactive user experience.

5.6 Database Implementation

The database implementation of the **Property Hub** platform was designed using **MySQL** in combination with **SQLAlchemy ORM** to ensure efficient data management, scalability, and optimized query execution [10] [20].

This approach provides a reliable relational structure while enabling seamless integration with the backend services

The database is responsible for storing and managing all core system data, including:

- User Information and Authentication Data
- Property Listings and Detailed Property Attributes
- User Interaction Data (e.g., views, clicks, searches)
- Saved and Favorite Properties

SQLAlchemy ORM is used as an abstraction layer between the application and the database, allowing developers to interact with database tables using Python objects instead of raw SQL queries. This improves development efficiency, maintainability, and reduces the risk of database-related errors.

The database schema is designed based on the Entity-Relationship Diagram (ERD) presented in Chapter 3, ensuring proper mapping between entities and well-defined relationships.

Database Design and Functionality

The database structure is optimized to support the core functionalities of the system, including:

- Efficient processing of recommendation algorithms
- Real-time tracking of user interactions and behavior
- Storage and retrieval of prediction history and results
- Fast querying for property search and filtering operations

Overall, the database design ensures data consistency, integrity, and high performance, which are essential for supporting the intelligent features of the **Property Hub** platform.

5.7 Machine Learning Integration

The machine learning components were fully integrated into the backend architecture of the **Property Hub** platform using **FastAPI** and RESTful APIs. This integration enables seamless communication between the backend services and the trained machine learning models, supporting real-time recommendations and property price predictions [12] [19] [8].

The integration process was designed to ensure high performance, scalability, and low-latency responses while maintaining system reliability and modularity.

ML Integration Workflow

The machine learning workflow within the system follows a structured pipeline that includes:

- Data Collection from the Relational Database
- Data Preprocessing and Cleaning
- Feature Engineering and Transformation
- Model Inference and Prediction Execution
- API Response Generation and Delivery

This pipeline ensures that all incoming user requests are processed efficiently and passed through the appropriate machine learning models to generate accurate outputs.

System Communication

The backend communicates directly with the trained machine learning models during both recommendation and prediction requests. When a user interacts with the system, relevant data is retrieved from the database, processed, and forwarded to the ML models for inference in real time.

Dynamic Learning Support

The integration architecture is designed to support dynamic user interaction processing, allowing the system to continuously adapt to new data patterns. This contributes to improving recommendation relevance and overall prediction accuracy over time.

Overall, the ML integration layer acts as a bridge between data storage, backend services, and intelligent models, ensuring smooth and efficient operation of the **Property Hub** platform.

5.8 Authentication and Security

Security is a critical component of the **Property Hub** platform, ensuring the protection of user data and secure access to system resources. The authentication system was implemented using **JSON Web Tokens (JWT)** to secure backend APIs and manage user sessions in a stateless and scalable manner [12].

The security design focuses on protecting sensitive data, preventing unauthorized access, and ensuring safe communication between the client and server.

Implemented Security Features

The system includes several key security mechanisms:

- JWT-Based Authentication
- Secure Password Hashing
- Protected API Endpoints
- Access Token Generation and Validation
- Secure Session Management

Password Security

User passwords are not stored in plain text. Instead, they are securely hashed using cryptographic hashing algorithms before being stored in the database. This approach significantly reduces the risk of data breaches and unauthorized password retrieval.

Token-Based Authentication

Upon successful login, a JWT access token is generated and assigned to the user. This token is then used to authenticate subsequent requests to protected API endpoints. Each request is validated by verifying the token's integrity and expiration before granting access to system resources.

Protected System Access

All sensitive operations within the system, such as accessing user-specific data, managing properties, and interacting with recommendation and prediction services, are protected using

authentication middleware. This ensures that only authorized users can interact with these functionalities.

Overall, the authentication and security layer ensures a secure, reliable, and scalable environment for the **Property Hub** platform.

5.9 System Testing

l correctness, and the reliability of machine learning components. Testing was conducted during both the implementation and integration phases to validate that all system modules operate as intended and interact seamlessly.

The testing process covered multiple levels, including functional testing, API testing, and machine learning model evaluation.

Functional Testing

Functional testing was performed to verify that all core system features operate correctly according to the system requirements. This included validating:

- User Registration and Login Functionality
- Property Management (CRUD Operations)
- Search and Filtering Mechanisms
- Recommendation Generation System
- Price Prediction Services via API

This phase ensured that all user-facing features behave correctly under expected usage scenarios.

API Testing

The RESTful APIs implemented using **FastAPI** were thoroughly tested using tools such as **Swagger UI** and **Postman** to ensure reliability and correctness.

The API testing process focused on validating:

- Request Validation and Input Handling
- Accuracy and Consistency of API Responses
- Authentication and Authorization Mechanisms
- Stability and Reliability of Endpoints

This ensured that all backend services are secure, stable, and capable of handling real-world requests efficiently.

Machine Learning Testing

The machine learning models used in the recommendation and price prediction modules were evaluated using standard performance metrics to assess their accuracy and effectiveness.

The evaluation metrics included:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 Score
- Precision
- Recall

These metrics were used to measure prediction accuracy and the effectiveness of the recommendation system. The results confirmed that the implemented models provide reliable and meaningful outputs that enhance user experience.

Summary

Overall, the testing phase confirmed that the **Property Hub** system is stable, functional, and capable of delivering accurate machine learning-driven recommendations and predictions while maintaining system reliability and performance.

5.10 Deployment and System Execution

The **Property Hub** platform was deployed using a modular client-server architecture designed to ensure scalability, maintainability, and efficient communication between all system components. This deployment strategy integrates all layers of the system into a unified environment that supports real-time processing and provides a seamless user experience.

Deployment Environment

The system is composed of the following core components:

- Frontend Application
- FastAPI Backend Server
- MySQL Database
- Machine Learning Services

Each component operates independently while communicating through well-defined RESTful APIs. This loose coupling enhances system flexibility, improves maintainability, and allows individual components to be updated or scaled without affecting the entire system.

System Execution Flow

During runtime, the backend server acts as the central processing layer of the system. It receives API requests from the frontend and coordinates the required operations by interacting with both the database and the machine learning services.

The execution flow includes:

- Retrieving and storing data from/to the database
- Processing user requests and business logic
- Invoking machine learning models for predictions and recommendations
- Returning structured responses to the frontend

This architecture ensures efficient system execution while maintaining low latency, high responsiveness, and reliable performance.

Supported System Features

The deployed system supports several real-time functionalities, including:

- Real-Time Property Recommendations
- Property Price Prediction Services
- User Authentication and Session Management
- Property Management Operations
- Recommendation Analytics and Monitoring

Deployment Characteristics

The deployment architecture was designed with the following key principles:

- **Scalability:** Ability to handle increasing numbers of users and requests
- **Maintainability:** Modular separation of system components for easier updates and debugging
- **Extensibility:** Support for future integration of additional machine learning models and analytics features
- **High Availability:** Reliable system communication and continuous service operation

Overall, the deployment strategy ensures that the **Property Hub** platform operates as a robust, scalable, and intelligent real estate system capable of supporting real-world usage scenarios effectively.

5.11 Challenges and Solutions

During the development and implementation of the **Property Hub** platform, several technical challenges were encountered across different system layers. These challenges were addressed using appropriate engineering and machine learning solutions to ensure system stability and performance.

Data Quality Challenges

One of the primary challenges was related to the quality of the collected dataset, as property data originated from multiple sources and contained inconsistencies, missing values, and duplicates.

Solution:

To address this issue, a comprehensive data preprocessing pipeline was implemented, which included:

- Handling Missing Values
- Removing Duplicate Records
- Feature Normalization and Scaling
- Data Validation and Consistency Checks

This ensured that the dataset used for training and system operations was clean and reliable.

Recommendation Accuracy Challenges

Achieving high-quality personalized recommendations was challenging, especially due to limited user interaction history during the early stages of system usage (cold-start problem).

Solution:

A **hybrid recommendation system** was implemented, combining both Content-Based Filtering and Collaborative Filtering techniques. This approach improved recommendation diversity and accuracy while reducing the impact of the cold-start problem.

Prediction Accuracy Challenges

Property price prediction was complex due to nonlinear relationships between property features and market prices.

Solution:

Advanced regression models, including **Random Forest Regressor** and **XGBoost**, were implemented to capture nonlinear patterns and improve prediction accuracy. Feature importance analysis was also utilized to enhance model interpretability.

Backend Integration Challenges

Integrating machine learning models with the backend APIs required efficient communication between system components and optimized response times for real-time predictions.

Solution:

The **FastAPI** framework was adopted due to its high performance and asynchronous capabilities. This enabled efficient integration of machine learning models and ensured scalable and low-latency API responses.

5.12 Summary

This chapter presented the detailed implementation of the **Property Hub** platform, covering frontend development, backend architecture, database design, machine learning modules, recommendation system, authentication and security mechanisms, system testing, and deployment strategy.

The implemented system successfully integrates intelligent recommendation services and property price prediction models within a scalable **FastAPI-based architecture**. It demonstrates the transformation of the proposed design into a fully functional intelligent real estate platform capable of delivering personalized recommendations, predictive analytics, and efficient property management services.

Overall, the implementation ensures a robust, scalable, and extensible system that meets the functional and technical requirements of the **Property Hub** project.

6. Results and Discussion

This chapter presents the experimental results, system outputs, and performance evaluation of the implemented **Property Hub** platform. It also discusses the effectiveness of the recommendation system, the accuracy of the property price prediction models, backend performance, and overall system usability.

The evaluation process focuses on assessing the ability of the system to deliver intelligent recommendations, accurate price predictions, and stable real-time performance while maintaining a responsive and user-friendly interface.

The obtained results demonstrate that the platform successfully integrates machine learning capabilities within a scalable web-based architecture, providing meaningful and efficient real estate services.

6.1 Recommendation System Results

The recommendation engine was evaluated using real and simulated user interaction data collected during system testing. The implemented hybrid recommendation system successfully generated personalized property suggestions by analyzing multiple factors, including:

- User Preferences
- Property Similarity
- Interaction History
- Saved Properties
- Behavioral Patterns

The results indicate that the system effectively improves user experience by reducing search effort and increasing the relevance of recommended properties. Recommendations were generated dynamically and ranked based on similarity scores and user preference alignment.

Activity Dashboard

#	PROPERTY	CITY	TYPE	OFFERING	PRICE	VIEWS	FAVS	CLICKS	CONTACT	SCORE	LAST SEEN
1	#5451945	alexandria	apartment	Rent	35,000 EGP	1	0	0	0	1	6/17/202
2	#65918971	cairo	apartment	Sale	10,000,000 EGP	1	0	0	0	1	6/17/202
3	#76506677	cairo	apartment	Sale	6,665,000 EGP	1	0	0	0	1	6/17/202
4	#6271434	cairo	apartment	Sale	7,875,000 EGP	1	0	0	0	1	6/15/202
5	#68782873	suez	apartment	Sale	3,480,000 EGP	1	0	0	0	1	6/14/202
6	#69312649	cairo	apartment	Sale	2,500,000 EGP	1	0	0	0	1	6/14/202
7	#64637108	giza	townhouse	Sale	8,000,000 EGP	1	0	0	0	1	6/14/202
8	#7406522	giza	villa	Sale	4,415,680 EGP	1	0	0	0	1	6/14/202
9	#64892118	red sea	apartment	Sale	11,200,000 EGP	1	0	0	0	1	6/14/202
10	#69838736	cairo	apartment	Sale	12,000,000 EGP	1	0	0	0	1	6/14/202

The interaction for user

Recommended For You

Recommended for you



For Sale
6,900,000 EGP
 new cairo city, cairo
 3 2 148 m²



For Sale
6,800,000 EGP
 new cairo city, cairo
 3 3 140 m²



For Sale
6,800,000 EGP
 new cairo city, cairo
 3 3 140 m²



For Sale
7,780,000 EGP
 new cairo city, cairo
 3 3 145 m²



For Sale
6,760,000 EGP
 new cairo city, cairo
 3 3 145 m²



For Sale
6,750,000 EGP
 new cairo city, cairo
 3 2 155 m²



For Sale
6,790,000 EGP
 new cairo city, cairo
 3 3 190 m²



For Sale
7,350,000 EGP
 new cairo city, cairo
 2 3 171 m²

Results for recommendations system

6.2 Property Price Prediction Results

The price prediction module was evaluated using supervised regression models trained on real estate datasets. The system estimates property prices based on key features such as:

- Property Location
- Area Size
- Number of Bedrooms
- Number of Bathrooms
- Property Type
- Additional Property Attributes

Several regression models were tested during the development phase, including:

- Linear Regression
- Random Forest Regressor
- XGBoost Regressor

The results show that ensemble-based models, particularly Random Forest and XGBoost, outperformed Linear Regression due to their ability to capture complex nonlinear relationships in the data.

Smart Price Predictor

Get AI-powered property valuation with similar property recommendations.

📍 LOCATION

CITY *
alexandria

AREA
Select area

COMPOUND
Select compound

🏠 PROPERTY DETAILS

PROPERTY TYPE *
apartment

OFFER TYPE *
sale

AREA (M²) *
120

BEDROOMS *
3

BATHROOMS *
2

★ AMENITIES (optional)

balcony

built in wardrobes

central a/c

covered parking

kitchen appliances

lobby in building

maids room

private garden

private pool

security

shared gym

shared pool

shared spa

study

view of landmark

view of water

walk-in closet

Value in Price prediction

AI Estimated Market Price

7,511,731.08 EGP

alexandria

apartment

120 m²

3 bd · 2 ba

For Sale

1 amenities

Similar Properties

★ 80% match



For Sale

first residence steps from alexandri...

4,600,000 EGP

hay wasat, alexandria

apartment

3 Beds · 2 Baths · 140 m²

View Details →

★ 79% match



For Sale

apartment for sale in muruj smouha...

5,344,000 EGP

hay sharq, alexandria

apartment

3 Beds · 3 Baths · 130 m²

View Details →

★ 79% match



For Sale

apartment for sale in muruj smouha...

5,344,000 EGP

hay sharq, alexandria

apartment

3 Beds · 3 Baths · 130 m²

View Details →

★ 77% match



For Sale

apartment for sale in muruj smouha...

5,344,000 EGP

hay sharq, alexandria

apartment

3 Beds · 3 Baths · 130 m²

View Details →

Prediction the price with content based recommendations

6.2.1 Prediction Evaluation Metrics

The models were evaluated using the following performance metrics:

- Mean Absolute Error (MAE): 1,936,693 EGP
- Root Mean Squared Error (RMSE): 4,339,291 EGP
- Mean Absolute Percentage Error (MAPE): 31.93%
- R^2 Score: 0.864

The evaluation results confirmed that the selected models provide accurate and reliable price predictions, making them suitable for real-world property valuation support and decision-making.

6.3 Backend Performance Results

The backend system was evaluated in terms of response time, stability, and integration efficiency with machine learning services. The **FastAPI** framework demonstrated strong performance in handling concurrent requests and ensuring efficient system operations.

The backend achieved:

- Low API Response Time
- Stable Authentication and Authorization Processes
- Efficient Database Communication
- Smooth Integration with Machine Learning Models

The asynchronous nature of FastAPI contributed significantly to improving system throughput and responsiveness

6.4 User Interface Results

The frontend system was evaluated based on usability, responsiveness, and overall user experience. The implemented interface provided a smooth and intuitive interaction flow across all system functionalities.

The frontend successfully delivered:

- Responsive Property Browsing Experience
- Interactive Recommendation Display
- Efficient Search and Filtering Mechanisms
- Real-Time Price Prediction Interface
- User-Friendly Navigation Structure

The responsive design ensured compatibility across different devices and screen sizes, improving accessibility and usability.

6.5 Discussion

The experimental results demonstrate the effectiveness of integrating machine learning techniques within a real estate platform. The hybrid recommendation system significantly enhances personalization by combining user behavior analysis with property feature similarity.

The price prediction models provide accurate and reliable estimations that can assist users in making informed property decisions. Additionally, the integration of machine learning with a **FastAPI-based backend architecture** ensures scalability, modularity, and efficient real-time processing.

Overall, the system successfully combines recommendation systems, predictive analytics, and modern backend technologies to deliver a comprehensive intelligent real estate solution. The results confirm the feasibility and effectiveness of applying machine learning in real-world property platforms.

6.6 Summary

This chapter presented the experimental evaluation and discussion of the **Property Hub** platform, including recommendation system performance, price prediction accuracy, backend efficiency, and frontend usability.

The results confirm that the implemented system successfully delivers intelligent, scalable, and user-centric real estate services powered by machine learning and modern web technologies.

7. Conclusion and Future Work

This chapter presents the final conclusions of the **Property Hub** project and outlines potential future enhancements that could further improve system intelligence, scalability, and user experience.

The project aimed to develop an intelligent real estate platform that integrates recommendation systems, machine learning-based property price prediction, backend APIs, and modern web technologies into a unified and scalable architecture. The implemented system successfully achieved its objectives and demonstrated the effectiveness of applying artificial intelligence techniques in real estate applications.

7.1 Conclusion

The **Property Hub** platform successfully provides an intelligent and interactive environment for property browsing, personalized recommendations, and property price prediction.

The system integrates multiple modern technologies, including:

- FastAPI Backend Services
- Relational Database Management System (MySQL)
- Machine Learning Models
- Recommendation Systems
- JWT-Based Authentication
- RESTful API Architecture

The recommendation engine significantly enhances user experience by generating personalized property suggestions based on user behavior, preferences, and property similarity analysis.

The price prediction module successfully estimates property values using regression-based machine learning models and feature engineering techniques, providing users with meaningful insights for decision-making.

In addition, the modular backend architecture improves system scalability, maintainability, and integration flexibility. It also enables efficient communication between the frontend interface, database layer, and machine learning services.

Overall, the implemented platform demonstrates the successful application of artificial intelligence and predictive analytics in modern real estate systems.

7.2 Achieved Objectives

The main objectives of the **Property Hub** project were successfully achieved during the implementation phase. These include:

- Development of an intelligent real estate platform
- Implementation of a personalized recommendation system
- Development of property price prediction models
- Design and implementation of scalable FastAPI backend services
- Creation of RESTful APIs for system communication
- Implementation of secure JWT-based authentication
- Development of responsive frontend interfaces
- Integration of machine learning models with backend APIs
- Implementation of user interaction tracking and analytics

The system effectively combines machine learning capabilities with modern backend technologies to deliver a unified intelligent platform.

7.3 Future Work

Although the **Property Hub** platform achieves its core objectives, several enhancements can further improve its performance, intelligence, and scalability.

Advanced Recommendation Models

Future versions of the system can integrate deep learning-based recommendation models, such as neural collaborative filtering, to improve personalization accuracy and overcome limitations of traditional methods.

Mobile Application Development

Developing a dedicated mobile application would improve accessibility and increase user engagement across mobile devices.

Advanced Analytics Dashboard

Future enhancements may include more advanced visualization tools and analytics dashboards for administrators and real estate analysts.

Cloud Deployment and Scalability

Deploying the system using cloud-native technologies such as Docker and Kubernetes can improve scalability, availability, and automated deployment processes.

Deep Learning-Based Price Prediction

Replacing or enhancing current regression models with deep learning architectures may improve prediction accuracy, especially when using larger datasets.

AI Chatbot Integration

An intelligent chatbot or virtual assistant can be integrated to enhance user support and improve the property search experience.

7.4 Final Summary

The **Property Hub** project demonstrates the successful integration of machine learning techniques, recommendation systems, predictive analytics, and modern backend architectures within a unified real estate platform.

The system combines intelligent recommendation engines, price prediction services, FastAPI-based backend APIs, relational database management, and responsive frontend interfaces into a scalable and maintainable architecture.

The project highlights the significant role of artificial intelligence in enhancing user experience, improving property discovery, and supporting data-driven decision-making in the real estate domain.

The obtained results confirm the effectiveness of the implemented approach and provide a strong foundation for future intelligent real estate systems and AI-driven property management platforms.

8. References

The following references were used during the development, implementation, and research phases of the Property Hub project.

- [1] Aqarmap Official Website. Available: <https://aqarmap.com.eg>
- [2] Dubizzle Egypt Official Website. Available: <https://www.dubizzle.com.eg>
- [3] Zillow Research Data. Available: <https://www.zillow.com/research/data/>
- [4] Robert C. Martin, *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2002.
- [5] Yehuda Koren, Robert Bell, and Chris Volinsky, “Matrix Factorization Techniques for Recommender Systems,” *IEEE Computer Society*, vol. 42, no. 8, pp. 30–37, 2009.
- [6] Francesco Ricci, Lior Rokach, and Bracha Shapira, “Introduction to Recommender Systems Handbook,” Springer, 2011.
- [7] LightGBM (LGBMRegressor) Documentation. Available: [LightGBM Documentation](#)
- [8] Aurélien Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O’Reilly Media, 2019.
- [9] Tianqi Chen and Carlos Guestrin, “XGBoost: A Scalable Tree Boosting System,” *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [10] Martin Kleppmann, *Designing Data-Intensive Applications*, O’Reilly Media, 2017.
- [11] Gediminas Adomavicius and Alexander Tuzhilin, “Toward the Next Generation of Recommender Systems: A Survey of the State-of-the-Art and Possible Extensions,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 6, pp. 734–749, 2005.
- [12] FastAPI Documentation. Available: <https://fastapi.tiangolo.com/>
- [13] Bootstrap Documentation. Available: <https://getbootstrap.com/>
- [14] MySQL Documentation. Available: [MySQL Official Documentation](#)

[15] Trevor Hastie, Robert Tibshirani, and Jerome Friedman, *The Elements of Statistical Learning*, Springer, 2009.

[16] Property Finder Real Estate Platform. Available: [Property Finder Official Website](#)

[17] Pandas Documentation. Available: <https://pandas.pydata.org/>

[18] NumPy Documentation. Available: <https://numpy.org/>

[19] Scikit-learn Documentation. Available: <https://scikit-learn.org/>

[20] SQLAlchemy Documentation. Available: <https://docs.sqlalchemy.org/>

[21] PostgreSQL Documentation. Available: <https://www.postgresql.org/docs/>

9. Appendix A —Screenshots

This appendix contains screenshots of the implemented property hub interfaces and

System functionalities:

Included screenshots:

- Home page

PROPERTYHUB

PropertiesPrice PredictorProfileLogout

AF

ahmed farag

cds.ahmedfarag23882@gmail.com

01126516608

Member

Edit Profile

12

Views

3

Favorites

0

Map Clicks

2

Contacts

♥ Saved Properties

For Sale

1br sea view apartment sahl h...

4,769,487 EGP

📍 Cala, Sahl Hasheesh, Hurghada, Red Sea, red sea

For Sale

2br ground chalet ramla 133 s...

20,809,759 EGP

📍 Ramla, Ras Al Hekma, North Coast, north coast

For Sale

apt prime location facing nor...

6,000,000 EGP

📍 Mountain View iCity, 5th Settlement Compounds, The 5th Settlement, New Cairo City, Cairo, cairo

📊 Activity Dashboard

#	PROPERTY	CITY	TYPE	OFFERING	PRICE	👁 VIEWS	❤ FAVS	👁 CLICKS	📞 CONTACT	SCORE	LAST SEEN
1	#6271434	cairo	apartment	🏠 Sale	7,675,000 EGP	1	0	0	0	1	6/15/2026
2	#68782873	suez	apartment	🏠 Sale	3,480,000 EGP	1	0	0	0	1	6/14/2026
3	#69312649	cairo	apartment	🏠 Sale	2,500,000 EGP	1	0	0	0	1	6/14/2026
4	#64637108	giza	townhouse	🏠 Sale	8,000,000 EGP	1	0	0	0	1	6/14/2026
5	#7406522	giza	villa	🏠 Sale	4,415,680 EGP	1	0	0	0	1	6/14/2026
6	#64892118	red sea	apartment	🏠 Sale	11,200,000 EGP	1	0	0	0	1	6/14/2026
7	#69638736	cairo	apartment	🏠 Sale	12,000,000 EGP	1	0	0	0	1	6/14/2026
8	#76502593	red sea	chalet	🏠 Sale	7,500,000 EGP	1	0	0	0	1	6/13/2026
9	#76503864	cairo	apartment	🏠 Sale	16,000,000 EGP	1	0	0	1	11	6/13/2026
10	#76504563	red sea	apartment	🏠 Sale	4,769,490 EGP	1	1	0	1	16	6/13/2026

🏠 Recommended For You

Recommended for you

For Sale

6,900,000 EGP

📍 new cairo city, cairo

↔ 3 🏠 2 🌳 148 m²

For Sale

7,900,000 EGP

📍 new cairo city, cairo

↔ 3 🏠 3 🌳 160 m²

For Sale

7,780,000 EGP

📍 new cairo city, cairo

↔ 3 🏠 3 🌳 145 m²

For Sale

12,500,000 EGP

📍 new cairo city, cairo

↔ 2 🏠 2 🌳 136 m²

For Sale

7,350,000 EGP

📍 new cairo city, cairo

↔ 2 🏠 3 🌳 171 m²

For Sale

10,050,000 EGP

📍 new cairo city, cairo

↔ 3 🏠 3 🌳 135 m²

For Sale

7,376,000 EGP

📍 new cairo city, cairo

↔ 1 🏠 1 🌳 108 m²

For Sale

7,250,000 EGP

📍 new cairo city, cairo

↔ 2 🏠 3 🌳 126 m²

- Property listing page:

PROPERTYHUB

[Properties](#)
[Price Predictor](#)

Logout

Find Your **Perfect** Property

Explore verified properties across Egypt.

Search area...

Min (1000)

Max (99990000)

Property Type

City

1 - 13

1 - 10

Search

For Sale

For Rent

For Sale

apartment 175m for sale in installments over 12y

7,875,000 EGP

new cairo city, cairo

Type: apartment

For Sale

villa golf view 5 down paymnet qatari diar

35,000,000 EGP

new cairo city, cairo

Type: villa

For Sale

for sale apartment 2bd with good view

4,600,000 EGP

new cairo city, cairo

Type: apartment

For Sale

apt prime location facing north under market price

6,000,000 EGP

new cairo city, cairo

Type: apartment

For Sale

apt 2bed with dp 5 inst 12y cash disc up to 64

3,870,000 EGP

mostakbal city - future city, cairo

Type: apartment

For Sale

apartment 3bedroom at lowest price in new capital

4,590,000 EGP

new capital city, cairo

Type: apartment

- Property Details:




apartment 175m for sale in installments over 12y

 The Kora Residences, 6th Settlement Compound, The 6th Settlement, New Cairo City, Cairo : 01 8980 6371332

3 bedrooms
3 bathrooms
475 sq ft
apartment
for sale
condominium

7,875,000 EGP

Registration - Owner verified

AT Ahmed Farag
PropertyMART Verified Agent

+966522877877

Call
WhatsApp
Schedule

About This Property

apartment 175m for sale in installments over 12 years next to Hyde park with a reservation payment of 50000 includes the own garage 17th settlement new 17th consists of 3 bedrooms 3 bathrooms reception terrace prime location open view with a reservation payment of 50000 fully refundable total 10700000 egypt and installments over the longest repayment period of up to 12 years for more details please contact by phone or whatsapp group agents a group of real estate sales calls providing a 24-hour security and guarding team to ensure the safe growth the units inside the kora residences are fully secured by

Features & Amenities

 already	 balcony	 private garden
 storage unit	 view open	 security
 covered parking	 walk-in closet	 view of water
 view of landmark	 shared gym	 safety in building
 children's pool		

Location



[Open in Google Maps](#)

Recommended For You

Recommended by our AI engine - Based on your browsing history

Watched on: [New Cairo City](#) [+100% at 13% EGP](#) [Per Day](#) [In 3 hours in 1](#)



FOR SALE

fully finished apartment with around kitchen

6,900,000 EGP

1 bedroom only, 120 sq ft

[View](#) [For Sale](#) [3 beds](#)



FOR SALE

practical space prime loca overlooking view rrm

7,900,000 EGP

1 bedroom only, 120 sq ft

[View](#) [For Sale](#) [3 beds](#)



FOR SALE

sky left prime location facing north ready to move

7,700,000 EGP

1 bedroom only, 120 sq ft

[View](#) [For Sale](#) [3 beds](#)



FOR SALE

fully finished apartment market corner walk 84p

12,500,000 EGP

1 bedroom only, 120 sq ft

[View](#) [For Sale](#)



FOR SALE

apartment for sale in 8th square fully finished

7,350,000 EGP

1 bedroom only, 120 sq ft

[View](#) [For Sale](#) [3 beds](#)



FOR SALE

apartment fully finished at lowest price only 10p

10,050,000 EGP

1 bedroom only, 120 sq ft

[View](#) [For Sale](#) [3 beds](#)



FOR SALE

Iris ocean lake shore with access prime location

7,170,000 EGP

1 bedroom only, 120 sq ft

[View](#) [For Sale](#) [3 beds](#)



FOR SALE

apartment for resale at lowest price district 5

7,250,000 EGP

1 bedroom only, 120 sq ft

[View](#) [For Sale](#) [3 beds](#)

- Recommendation page:

Recommended For You

Personalised by our ML engine — based on your browsing history

Matched on: new cairo city ±30% of 7,875,000 EGP For Sale 3 beds (±1)

99% match

FOR SALE

fully finished apartment with acesand kitchen

6,900,000 EGP

new cairo city, cairo · 3 beds · 148 m²

cairo For Sale 3 beds

98% match

FOR SALE

practical space prime loca stunning view rtm

7,900,000 EGP

new cairo city, cairo · 3 beds · 160 m²

cairo For Sale 3 beds

90% match

FOR SALE

sky loft prime location facing north ready to move

7,780,000 EGP

new cairo city, cairo · 3 beds · 145 m²

cairo For Sale 3 beds

89% match

FOR SALE

fully finished apartment markez crescent walk 8dp

12,500,000 EGP

new cairo city, cairo · 2 beds · 136 m²

cairo For Sale

88% match

FOR SALE

apartment for sale in fifth square fully finished

7,350,000 EGP

new cairo city, cairo · 2 beds · 171 m²

cairo For Sale

Similar price

88% match

FOR SALE

apartment fully finished at lowest price only rod p

10,050,000 EGP

new cairo city, cairo · 3 beds · 135 m²

cairo For Sale 3 beds

88% match

FOR SALE

iris swan lake 1hr with aces prime location

7,376,000 EGP

new cairo city, cairo · 1 beds · 108 m²

cairo For Sale

Similar price

87% match

FOR SALE

apartment for resale at lowest price district 5

7,250,000 EGP

new cairo city, cairo · 2 beds · 126 m²

cairo For Sale

Similar price

87% match

FOR SALE

strong invet opportunity zero dp over 10 years

21,900,000 EGP

new cairo city, cairo · 3 beds · 140 m²

cairo For Sale 3 beds

87% match

FOR SALE

3br apartment for sale only 5 dpinstgrand lane

7,833,000 EGP

new cairo city, cairo · 3 beds · 157 m²

cairo For Sale 3 beds

- Price Prediction interface “with prediction result”:

Smart Price Predictor

Get AI-powered property valuation with similar property recommendations.

📍 LOCATION

CITY *

cairo

AREA

Select area

COMPOUND

Select compound

🏠 PROPERTY DETAILS

PROPERTY TYPE *

apartment

OFFER TYPE *

sale

AREA (M²) *

120

BEDROOMS *

3

BATHROOMS *

2

🌿 AMENITIES (optional)

balcony

built in wardrobes

central a/c

covered parking

kitchen appliances

lobby in building

maids room

private garden

private pool

security

shared gym

shared pool

shared spa

study

view of landmark

view of water

walk-in closet

📝 DESCRIPTION (optional)

Describe the property to improve prediction accuracy...

🔮 Predict Price

AI Estimated Market Price

8,177,394.46 EGP

📍 cairo

🏠 apartment

📏 120 m²

🛏 3 bd • 🚿 2 ba

💡 For Sale

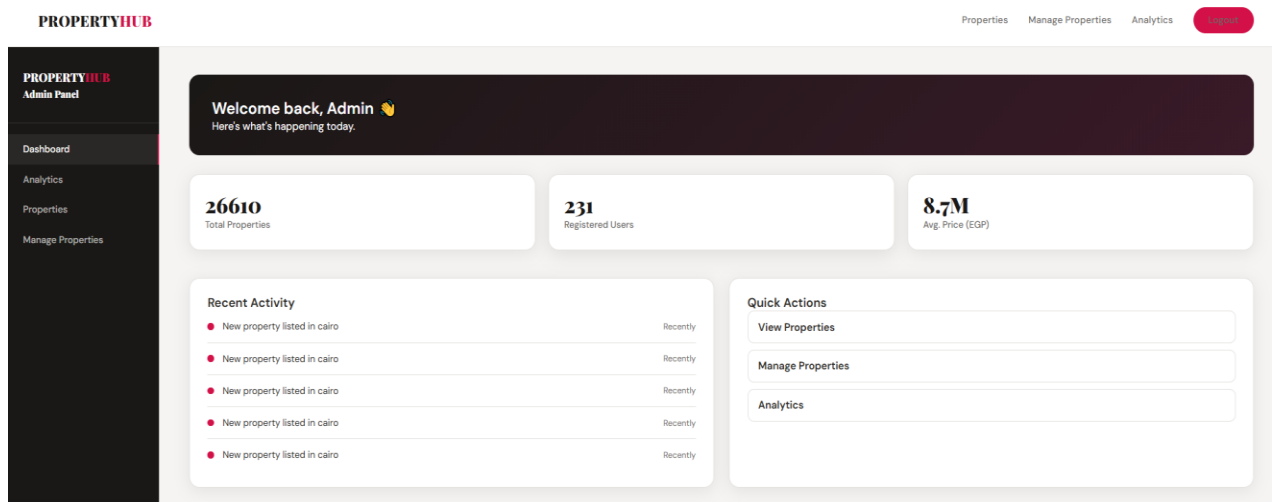
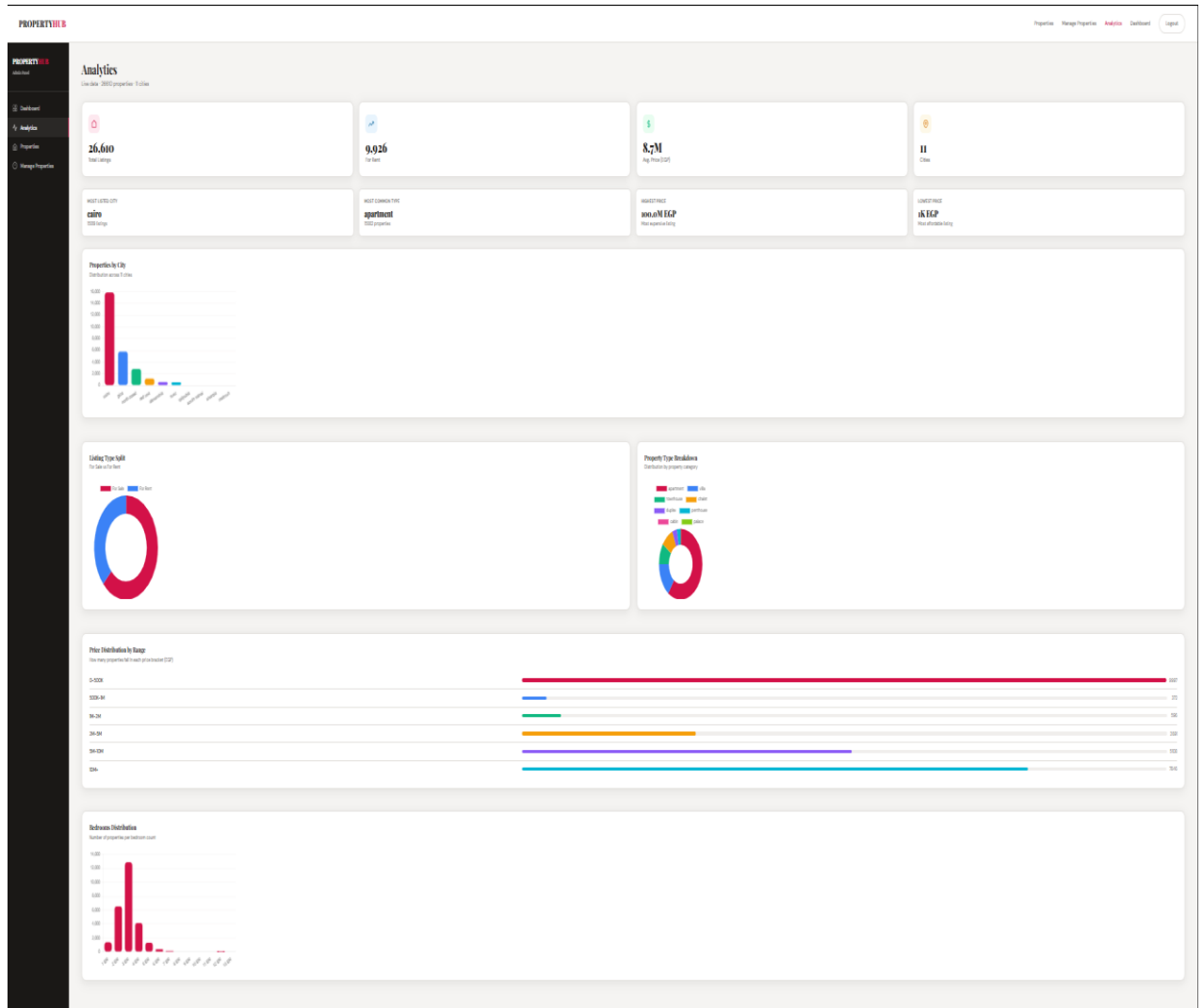
🌿 2 amenities

Similar Properties

★ 79% match

★ 75% match

- Analytics Dashboard:



- Registration and Login pages:

PROPERTYHUB

Hello, Welcome Back!

Sign in to your account or create a new one

[USER](#) [ADMIN](#) [REGISTER](#)

EMAIL ADDRESS

you@example.com

PASSWORD

[Forgot password?](#)

Sign in as User

PROPERTYHUB

Hello, Welcome Back!

Sign in to your account or create a new one

[USER](#) [ADMIN](#) [REGISTER](#)

FIRST NAME

John

LAST NAME

Doe

EMAIL ADDRESS

you@example.com

PHONE NUMBER

01000000000

PASSWORD

CONFIRM PASSWORD

Create Account

